

Assessment Test

1. What is the result of the following class? (Choose all that apply)

```
1: public class _C {  
2:     private static int $;  
3:     public static void main(String[] main) {  
4:         String a_b;  
5:         System.out.print($);  
6:         System.out.print(a_b);  
7:     } }
```

- A. Compiler error on line 1.
- B. Compiler error on line 2.
- C. Compiler error on line 4.
- D. Compiler error on line 5.
- E. **Compiler error on line 6.**
- F. `0null`
- G. `nullnull`

2. What is the result of the following code?

```
String s1 = "Java";  
String s2 = "Java";  
StringBuilder sb1 = new StringBuilder();  
sb1.append("Ja").append("va");  
System.out.println(s1 == s2);  
System.out.println(s1.equals(s2));  
System.out.println(sb1.toString() == s1);  
System.out.println(sb1.toString().equals(s1));
```

- A. `true` is printed out exactly once.
- B. `true` is printed out exactly twice.
- C. **`true` is printed out exactly three times.**
- D. `true` is printed out exactly four times.
- E. The code does not compile.

3. What is the output of the following code? (Choose all that apply)

```
1: interface HasTail { int getTailLength(); }  
2: abstract class Puma implements HasTail {  
3:     protected int getTailLength() {return 4;}  
4: }  
5: public class Cougar extends Puma {
```

```
6:   public static void main(String[] args) {
7:       Puma puma = new Puma();
8:       System.out.println(puma.getTailLength());
9:   }
10:
11:   public int getTailLength(int length) {return 2;}
12: }
```

A. 2

B. 4

C. The code will not compile because of line 3.

D. The code will not compile because of line 5.

E. The code will not compile because of line 7.

F. The code will not compile because of line 11.

G. The output cannot be determined from the code provided.

4. What is the output of the following program?

```
1: public class FeedingSchedule {
2:   public static void main(String[] args) {
3:       boolean keepGoing = true;
4:       int count = 0;
5:       int x = 3;
6:       while(count++ < 3) {
7:           int y = (1 + 2 * count) % 3;
8:           switch(y) {
9:               default:
10:                  case 0: x -= 1; break;
11:                  case 1: x += 5;
12:           }
13:       }
14:       System.out.println(x);
15: } }
```

A. 4

B. 5

C. 6

D. 7

E. 13

F. The code will not compile because of line 7.

5. What is the output of the following code snippet?

```
13: System.out.print("a");  
14: try {  
15:     System.out.print("b");  
16:     throw new IllegalArgumentException();  
17: } catch (RuntimeException e) {  
18:     System.out.print("c");  
19: } finally {  
20:     System.out.print("d");  
21: }  
22: System.out.print("e");
```

- A. abe
- B. abce
- C. abde
- D. abcde
- E. The code does not compile.
- F. An uncaught exception is thrown.

6. What is the result of the following program?

```
1: public class MathFunctions {  
2:     public static void addToInt(int x, int amountToAdd) {  
3:         x = x + amountToAdd;  
4:     }  
5:     public static void main(String[] args) {  
6:         int a = 15;  
7:         int b = 10;  
8:         MathFunctions.addToInt(a, b);  
9:         System.out.println(a);    } }
```

- A. 10
- B. 15
- C. 25
- D. Compiler error on line 3.
- E. Compiler error on line 8.
- F. None of the above.

7. What is the result of the following code?

```
int[] array = {6,9,8};  
List<Integer> list = new ArrayList<>();
```

```
list.add(array[0]);  
list.add(array[2]);  
list.set(1, array[1]);  
list.remove(0);  
System.out.println(list);
```

- A. [8]
 - B. [9]
 - C. Something like [Ljava.lang.String;@160bc7c0
 - D. An exception is thrown.
 - E. The code does not compile.
8. What is the output of the following code?
- ```
1: public class Deer {
2: public Deer() { System.out.print("Deer"); }
3: public Deer(int age) { System.out.print("DeerAge"); }
4: private boolean hasHorns() { return false; }
5: public static void main(String[] args) {
6: Deer deer = new Reindeer(5);
7: System.out.println(", "+deer.hasHorns());
8: }
9: }
10: class Reindeer extends Deer {
11: public Reindeer(int age) { System.out.print("Reindeer"); }
12: public boolean hasHorns() { return true; }
13: }
```
- A. DeerReindeer,false
  - B. DeerReindeer,true
  - C. ReindeerDeer,false
  - D. ReindeerDeer,true
  - E. DeerAgeReindeer,false
  - F. DeerAgeReindeer,true
  - G. The code will not compile because of line 7.
  - H. The code will not compile because of line 12.
9. Which of the following statements are true? (Choose all that apply)
- A. Checked exceptions are intended to be thrown by the JVM (and not the programmer).
  - B. Checked exceptions are required to be caught or declared.
  - C. Errors are intended to be thrown by the JVM (and not the programmer).
  - D. Errors are required to be caught or declared.
  - E. Runtime exceptions are intended to be thrown by the JVM (and not the programmer).
  - F. Runtime exceptions are required to be caught or declared.

**10.** Which are true of the following code? (Choose all that apply)

```
1: import java.util.*;
2: public class Grasshopper {
3: public Grasshopper(String n) {
4: name = n;
5: }
6: public static void main(String[] args) {
7: Grasshopper one = new Grasshopper("g1");
8: Grasshopper two = new Grasshopper("g2");
9: one = two;
10: two = null;
11: one = null;
12: }
13: private String name; }
```

- A.** Immediately after line 9, no grasshopper objects are eligible for garbage collection.
- B.** Immediately after line 10, no grasshopper objects are eligible for garbage collection.
- C.** Immediately after line 9, only one grasshopper object is eligible for garbage collection.
- D.** Immediately after line 10, only one grasshopper object is eligible for garbage collection.
- E.** Immediately after line 11, only one grasshopper object is eligible for garbage collection.
- F.** The code compiles.
- G.** The code does not compile.

**11.** What is the output of the following program?

```
1: public class FeedingSchedule {
2: public static void main(String[] args) {
3: int x = 5, j = 0;
4: OUTER: for(int i=0; i<3;)
5: INNER: do {
6: i++; x++;
7: if(x > 10) break INNER;
8: x += 4;
9: j++;
10: } while(j <= 2);
11: System.out.println(x);
12: } }
```

- A.** 10
- B.** 12
- C.** 13
- D.** 17
- E.** The code will not compile because of line 4.
- F.** The code will not compile because of line 6.

**12.** What is the result of the following program?

```
1: public class Egret {
2: private String color;
3: public Egret() {
4: this("white");
5: }
6: public Egret(String color) {
7: color = color;
8: }
9: public static void main(String[] args) {
10: Egret e = new Egret();
11: System.out.println("Color:" + e.color);
12: }
13: }
```

- A. Color:
- B. Color:null
- C. Color:White
- D. Compiler error on line 4.
- E. Compiler error on line 10.
- F. Compiler error on line 11.

**13.** What is the output of the following program?

```
1: public class BearOrShark {
2: public static void main(String[] args) {
3: int luck = 10;
4: if((luck>10 ? luck++: --luck)<10) {
5: System.out.print("Bear");
6: } if(luck<10) System.out.print("Shark");
7: } }
```

- A. Bear
- B. Shark
- C. BearShark
- D. The code will not compile because of line 4.
- E. The code will not compile because of line 6.
- F. The code compiles without issue but does not produce any output.

**14.** Assuming we have a valid, non-null HenHouse object whose value is initialized by the blank line shown here, which of the following are possible outputs of this application? (Choose all that apply)

```
1: class Chicken {}
2: interface HenHouse { public java.util.List<Chicken> getChickens(); }
3: public class ChickenSong {
```

```

4: public static void main(String[] args) {
5: HenHouse house = -----
6: Chicken chicken = house.getChickens().get(0);
7: for(int i=0; i<house.getChickens().size();
8: chicken = house.getChickens().get(i++)) {
9: System.out.println("Cluck");
10: } } }

```

- A. The code will not compile because of line 6.
  - B. The code will not compile because of lines 7–8.
  - C. The application will compile but not produce any output.
  - D. The application will output Cluck exactly once.
  - E. The application will output Cluck more than once.
  - F. The application will compile but produce an exception at runtime.
15. Which of the following statements can be inserted in the blank line so that the code will compile successfully? (Choose all that apply)

```

public interface CanSwim {}
public class Amphibian implements CanSwim {}
class Tadpole extends Amphibian {}
public class FindAllTadPole {
 public static void main(String[] args) {
 List<Tadpole> tadpoles = new ArrayList<Tadpole>();
 for(Amphibian amphibian : tadpoles) {
 ----- tadpole = amphibian;
 } } }

```

- A. CanSwim
  - B. Long
  - C. Amphibian
  - D. Tadpole
  - E. Object
16. What individual changes, if any, would allow the following code to compile? (Choose all that apply)
- ```

1: public interface Animal { public default String getName() { return null; } }
2: interface Mammal { public default String getName() { return null; } }
3: abstract class Otter implements Mammal, Animal {}

```
- A. The code compiles without issue.
 - B. Remove the default method modifier and method implementation on line 1.
 - C. Remove the default method modifier and method implementation on line 2.
 - D. Remove the default method modifier and method implementation on lines 1 and 2.
 - E. Change the return value on line 1 from null to "Animal".

- F. Override the `getName()` method with an abstract method in the `Otter` class.
- G. Override the `getName()` method with a concrete method in the `Otter` class.
17. Which of the following lines can be inserted at line 11 to print true? (Choose all that apply)
- ```
10: public static void main(String[] args) {
11: // INSERT CODE HERE
12: }
13: private static boolean test(Predicate<Integer> p) {
14: return p.test(5);
15: }
```
- A. `System.out.println(test(i -> i == 5));`
- B. `System.out.println(test(i -> {i == 5;}));`
- C. `System.out.println(test((i) -> i == 5));`
- D. `System.out.println(test((int i) -> i == 5);`
- E. `System.out.println(test((int i) -> {return i == 5;}));`
- F. `System.out.println(test((i) -> {return i == 5;}));`
18. Which of the following print out a date representing April 1, 2015? (Choose all that apply)
- A. `System.out.println(LocalDate.of(2015, Calendar.APRIL, 1));`
- B. `System.out.println(LocalDate.of(2015, Month.APRIL, 1));`
- C. `System.out.println(LocalDate.of(2015, 3, 1));`
- D. `System.out.println(LocalDate.of(2015, 4, 1));`
- E. `System.out.println(new LocalDate(2015, 3, 1));`
- F. `System.out.println(new LocalDate(2015, 4, 1));`
19. Bytecode is in a file with which extension?
- A. `.bytecode`
- B. `.bytes`
- C. `.class`
- D. `.exe`
- E. `.javac`
- F. `.java`
20. Which of the following are checked exceptions? (Choose all that apply)
- A. `Exception`
- B. `IllegalArgumentException`
- C. `IOException`
- D. `NullPointerException`
- E. `NumberFormatException`
- F. `StackOverflowError`

# Answers to Assessment Test

1. E. Option E is correct because local variables require assignment before referencing them. Option D is incorrect because class and instance variables have default values and allow referencing. `a_b` defaults to a null value. Options A, B, and C are incorrect because identifiers may begin with a letter, underscore, or dollar sign. Options F and G are incorrect because the code does not compile. If `a_b` was an instance variable, the code would compile and output `0null`. For more information, see Chapter 1.
2. C. String literals are used from the string pool. This means that `s1` and `s2` refer to the same object and are equal. Therefore, the first two print statements print true. The third print statement prints false because `toString()` uses a method to compute the value and it is not from the string pool. The final print statement again prints true because `equals()` looks at the values of String objects. For more information, see Chapter 3.
3. C, D, E. First, the method `getTailLength()` in the interface `HasTail` is assumed to be public, since it is part of an interface. The implementation of the method on line 3 is therefore an invalid override, as `protected` is a more restrictive access modifier than public, so option C is correct. Next, the class `Cougar` implements an overloaded version of `getTailLength()`, but since the declaration in the parent class `Puma` is invalid, it needs to implement a public version of the method. Since it does not, the declaration of `Puma` is invalid, so option D is correct. Option E is incorrect, since `Puma` is marked abstract and cannot be instantiated. The overloaded method on line 11 is declared correctly, so option F is not correct. Finally, as the code has multiple compiler errors, options A, B, and G can be eliminated. For more information, see Chapter 5.
4. C. The code compiles and runs without issue; therefore, option F is incorrect. This type of problem is best examined one loop iteration at a time:
  - The loop continues as count loop expression evaluates to  $0 < 3$ , which is true, with `y` taking a new value of 1. The value of `y` is set to:
 

```
y = (1 + 2 * 1) % 3
 = (1 + 2) % 3
 = 3 % 3
 = 0
```
  - The first case block is called and the value of `x` is then set to:
 

```
x = 3 - 1 = 2
```
  - The loop continues as count loop expression evaluates to  $1 < 3$ , which is true, with `y` taking a new value of 2. The value of `y` is set to:
 

```
y = (1 + 2 * 2) % 3
 = (1 + 4) % 3
 = 4 % 3
 = 2
```

- The default block is called and the value of `x` is set to:  

$$x = 2 - 1 = 1$$
  - The loop continues as the count loop expression evaluates to  $2 < 3$ , which is true, with `y` taking a new value of 3. The value of `y` is set to:  

$$\begin{aligned} y &= (1 + 2 * 3) \% 3 \\ &= (1 + 6) \% 3 \\ &= 7 \% 3 \\ &= 1 \end{aligned}$$
  - The second case block is called and the value of `x` is then set to:  

$$x = 1 + 5 = 6$$
  - The loop ends as the count loop expression evaluates to  $3 < 3$ , with `y` also taking a new value of 4. The most recent value of `x`, 6, is output, so the answer is option C. For more information, see Chapter 2.
5. D. The code starts running and prints `a` and `b` on lines 13 and 15. Line 16 throws an exception, which is caught on line 17. After line 18 prints `c`, the `finally` block is run and `d` is printed. Then the `try` statement ends and `e` is printed on line 22. For more information, see Chapter 6.
  6. B. The code compiles successfully, so options D and E are incorrect. The value of `a` cannot be changed by the `addToInt` method, no matter what the method does, because only a copy of the variable is passed into the parameter `x`. Therefore, `a` does not change and the output on line 9 is 15. For more information, see Chapter 4.
  7. B. The array is allowed to use an anonymous initializer because it is in the same line as the declaration. The `ArrayList` uses the diamond operator allowed since Java 7. This specifies the type matches the one on the left without having to re-type it. After adding the two elements, `list` contains `[6, 8]`. We then replace the element at index 1 with 9, resulting in `[6, 9]`. Finally, we remove the element at index 0, leaving `[9]`. Option C is incorrect because arrays output something like that rather than an `ArrayList`. For more information, see Chapter 3.
  8. A. The code compiles and runs without issue, so options G and H are incorrect. First, the `Reindeer` object is instantiated using the constructor that takes an `int` value. Since there is no explicit call to the parent constructor, the default no-argument `super()` is inserted as the first line of the constructor. The output is then `Deer`, followed by `Reindeer` in the child constructor, so only options A and B can be correct. Next, the method `hasHorns()` looks like an overridden method, but it is actually a hidden method since it is declared `private` in the parent class. Because the hidden method is referenced in the parent class, the parent version is used, so the code outputs `false`, and option A is the correct answer.
  9. B, C. Only checked exceptions are required to be handled (caught) or declared. Runtime exceptions are commonly thrown by both the JVM and programmer code. Checked exceptions are usually thrown by programmer code. Errors are intended to be

thrown by the JVM. While a programmer could throw one, this would be a horrible practice. For more information, see Chapter 6.

10. C, D, F. Immediately after line 9, only Grasshopper `g1` is eligible for garbage collection since both one and two point to Grasshopper `g2`. Immediately after line 10, we still only have Grasshopper `g1` eligible for garbage collection. Reference one points to `g1` and reference two is `null`. Immediately after line 11, both Grasshopper objects are eligible for garbage collection since both one and two point to `null`. The code does compile. Although it is traditional to declare instance variables early in the class, you don't have to. For more information, see Chapter 1.
11. B. The code compiles and runs without issue; therefore, options E and F are incorrect. This type of problem is best examined one loop iteration at a time:
  - On the first iteration of the outer loop `i` is 0, so the loop continues.
  - On the first iteration of the inner loop, `i` is updated to 1 and `x` to 6. The `if`-then statement branch is not executed, and `x` is increased to 10 and `j` to 1.
  - On the second iteration of the inner loop (since `j = 1` and `1 <= 2`), `i` is updated to 2 and `x` to 11. At this point, the `if`-then branch will evaluate to true for the remainder of the program run, which causes the flow to break out of the inner loop each time it is reached.
  - On the second iteration of the outer loop (since `i = 2`), `i` is updated to 3 and `x` to 12. As before, the inner loop is broken since `x` is still greater than 10.
  - On the third iteration of the outer loop, the outer loop is broken, as `i` is already not less than 3. The most recent value of `x`, 12, is output, so the answer is option B. For more information, see Chapter 2.
12. B. Line 10 calls the constructor on lines 3–5. That constructor calls the other constructor. However, the constructor on lines 6–8 assigns the method parameter to itself, which leaves the `color` instance variable on line 2 set to its default value of `null`. For more information, see Chapter 4.
13. C. The code compiles and runs without issue, so options D and E are correct. Remember that only one of the right-hand ternary expressions will be evaluated at runtime. Since `luck` is not less than 10, the second expression, `--luck`, will be evaluated, and since the pre-increment operator was used, the value returned will be 9, which is less than 10. So the first `if`-then statement will be visited and Bear will be output. Notice there is no `else` statement on line 6. Since `luck` is still less than 10, the second `if`-then statement will also be reached and Shark will be output; therefore, the correct answer is option C. For more information, see Chapter 2.
14. D, E, F. The code compiles without issue, so options A and B are incorrect. If `house.getChickens()` returns an array of one element, the code will output Cluck once, so option D is correct. If `house.getChickens()` returns an array of multiple elements, the code will output Cluck once for each element in the array, so option E is correct. Alternatively, if `house.getChickens()` returns an array of zero elements, then the code will

throw an `IndexOutOfBoundsException` on the call to `house.getChickens().get(0)`; therefore, option C is not possible and option F is correct. The code will also throw an exception if the array returned by `house.getChickens()` is null, so option F is possible under multiple circumstances. For more information, see Chapter 2.

15. A, C, E. The for-each loop automatically casts each `Tadpole` object to an `Amphibian` reference, which does not require an explicit cast because `Tadpole` is a subclass of `Amphibian`. From there, any parent class or interface that `Amphibian` inherits from is permitted without an explicit cast. This includes `CanSwim`, the interface `Amphibian` implements, and `Object`, which all classes extend from, so options A and E are correct. Option C is also correct since the reference is being cast to the same type, so no explicit cast is required. Option B is incorrect, since `Long` is not a parent of `Amphibian`. Option D is incorrect as well, although an explicit cast to `Tadpole` on the right-hand side of the expression would be required to allow the code to compile. For more information, see Chapter 5.
16. D, F, G. The code does not compile, since a class cannot inherit two interfaces that both define default methods with the same signature, unless the class implementing the interfaces overrides it with an abstract or concrete method. Therefore, option A is incorrect and options F and G are correct. The alternate approach is to make the `getName()` method abstract in the interfaces, because an interface may inherit two abstract methods with the same signature. The change must be made to both interfaces, though, so options B and C are incorrect if taken individually, and option D is correct since the changes are taken together. For more information, see Chapter 5.
17. A, C, F. The only functional programming interface you need to memorize for the exam is `Predicate`. It takes a single parameter and returns a `boolean`. Lambda expressions with one parameter are allowed to omit the parentheses around the parameter list, making options A and C correct. The return statement is optional when a single statement is in the body, making option F correct. Option B is incorrect because a return statement must be used if braces are included around the body. Options D and E are incorrect because the type is `Integer` in the predicate and `int` in the lambda. Autoboxing works for collections not inferring predicates. If these two were changed to `Integer`, they would be correct. For more information, see Chapter 4.
18. B, D. The new date APIs added in Java 8 use static methods rather than a constructor to create a new date, making options E and F incorrect. The months are indexed starting with 1 in these APIs, making options A and C incorrect. Option A uses the old `Calendar` constants which are indexed from 0. Therefore, options B and D are correct. For more information, see Chapter 3.
19. C. Files with the `.java` extension contain the Java source code and are compiled to files with the `.class` extension that contain the bytecode. For more information, see Chapter 1.
20. A, C. Option A is the exception base class, which is a checked exception. Options B, D, and E extend `RuntimeException` directly or indirectly and therefore are unchecked exceptions. Option F is a throwable and not an exception, and so should not be caught or declared. For more information, see Chapter 6.

local variables. Instance variables are the nonstatic fields of your class. Class variables are the static fields within a class. Local variables are declared within a method.

For some class elements, order matters within the file. The package statement comes first if present. Then comes the import statements if present. Then comes the class declaration. Fields and methods are allowed to be in any order within the class.

Garbage collection is responsible for removing objects from memory when they can never be used again. An object becomes eligible for garbage collection when there are no more references to it or its references have all gone out of scope. The `finalize()` method will run once for each object if/when it is first garbage collected.

Java code is object oriented, meaning all code is defined in classes. Access modifiers allow classes to encapsulate data. Java is platform independent, compiling to bytecode. It is robust and simple by not providing pointers or operator overloading. Finally, Java is secure because it runs inside a virtual machine.

## Exam Essentials

**Be able to write code using a `main()` method.** A `main()` method is usually written as `public static void main(String[] args)`. Arguments are referenced starting with `args[0]`. Accessing an argument that wasn't passed in will cause the code to throw an exception.

**Understand the effect of using packages and imports.** Packages contain Java classes. Classes can be imported by class name or wildcard. Wildcards do not look at subdirectories. In the event of a conflict, class name imports take precedence.

**Be able to recognize a constructor.** A constructor has the same name as the class. It looks like a method without a return type.

**Be able to identify legal and illegal declarations and initialization.** Multiple variables can be declared and initialized in the same statement when they share a type. Local variables require an explicit initialization; others use the default value for that type. Identifiers may contain letters, numbers, \$, or \_. Identifiers may not begin with numbers. Numeric literals may contain underscores between two digits and begin with 1–9, 0, 0x, 0X, 0b, and 0B.

**Be able to determine where variables go into and out of scope.** All variables go into scope when they are declared. Local variables go out of scope when the block they are declared in ends. Instance variables go out of scope when the object is garbage collected. Class variables remain in scope as long as the program is running.

**Be able to recognize misplaced statements in a class.** Package and import statements are optional. If present, both go before the class declaration in that order. Fields and methods are also optional and are allowed in any order within the class declaration.

**Know how to identify when an object is eligible for garbage collection.** Draw a diagram to keep track of references and objects as you trace the code. When no arrows point to a box (object), it is eligible for garbage collection.

# Review Questions

1. Which of the following are valid Java identifiers? (Choose all that apply)

- A. A\$B
- B. \_helloWorld
- C. true
- D. java.lang
- E. Public
- F. 1980\_s

2. What is the output of the following program?

```
1: public class WaterBottle {
2: private String brand;
3: private boolean empty;
4: public static void main(String[] args) {
5: WaterBottle wb = new WaterBottle();
6: System.out.print("Empty = " + wb.empty);
7: System.out.print(", Brand = " + wb.brand);
8: } }
```

- A. Line 6 generates a compiler error.
- B. Line 7 generates a compiler error.
- C. There is no output.
- D. Empty = false, Brand = null
- E. Empty = false, Brand =
- F. Empty = null, Brand = null

3. Which of the following are true? (Choose all that apply)

```
4: short numPets = 5;
5: int numGrains = 5.6;
6: String name = "Scruffy";
7: numPets.length();
8: numGrains.length();
9: name.length();
```

- A. Line 4 generates a compiler error.
- B. Line 5 generates a compiler error.
- C. Line 6 generates a compiler error.
- D. Line 7 generates a compiler error.
- E. Line 8 generates a compiler error.

**F.** Line 9 generates a compiler error.

**G.** The code compiles as is.

- 4.** Given the following class, which of the following is true? (Choose all that apply)

```
1: public class Snake {
2:
3: public void shed(boolean time) {
4:
5: if (time) {
6:
7: }
8: System.out.println(result);
9:
10: }
11: }
```

**A.** If `String result = "done";` is inserted on line 2, the code will compile.

**B.** If `String result = "done";` is inserted on line 4, the code will compile.

**C.** If `String result = "done";` is inserted on line 6, the code will compile.

**D.** If `String result = "done";` is inserted on line 9, the code will compile.

**E.** None of the above changes will make the code compile.

- 5.** Given the following classes, which of the following can independently replace `INSERT IMPORTS HERE` to make the code compile? (Choose all that apply)

```
package aquarium;
public class Tank { }
```

```
package aquarium.jellies;
public class Jelly { }
```

```
package visitor;
INSERT IMPORTS HERE
public class AquariumVisitor {
 public void admire(Jelly jelly) { } }
```

**A.** `import aquarium.*;`

**B.** `import aquarium.*.Jelly;`

**C.** `import aquarium.jellies.Jelly;`

**D.** `import aquarium.jellies.*;`

**E.** `import aquarium.jellies.Jelly.*;`

**F.** None of these can make the code compile.

6. Given the following classes, what is the maximum number of imports that can be removed and have the code still compile?

```
package aquarium; public class Water { }
```

```
package aquarium;
import java.lang.*;
import java.lang.System;
import aquarium.Water;
import aquarium.*;
public class Tank {
 public void print(Water water) {
 System.out.println(water); } }
```

- A. 0
  - B. 1
  - C. 2
  - D. 3
  - E. 4
  - F. Does not compile.
7. Given the following classes, which of the following snippets can be inserted in place of INSERT IMPORTS HERE and have the code compile? (Choose all that apply)

```
package aquarium;
public class Water {
 boolean salty = false;
}
package aquarium.jellies;
public class Water {
 boolean salty = true;
}
package employee;
 INSERT IMPORTS HERE
public class WaterFiller {
 Water water;
}
```

- A. `import aquarium.*;`
- B. `import aquarium.Water;`  
`import aquarium.jellies.*;`
- C. `import aquarium.*;`  
`import aquarium.jellies.Water;`

- D. `import aquarium.*;`  
`import aquarium.jellies.*;`
  - E. `import aquarium.Water;`  
`import aquarium.jellies.Water;`
  - F. None of these imports can make the code compile.
8. Given the following class, which of the following calls print out Blue Jay? (Choose all that apply)
- ```
public class BirdDisplay {  
    public static void main(String[] name) {  
        System.out.println(name[1]);  
    }  
}
```
- A. `java BirdDisplay Sparrow Blue Jay`
 - B. `java BirdDisplay Sparrow "Blue Jay"`
 - C. `java BirdDisplay Blue Jay Sparrow`
 - D. `java BirdDisplay "Blue Jay" Sparrow`
 - E. `java BirdDisplay.class Sparrow "Blue Jay"`
 - F. `java BirdDisplay.class "Blue Jay" Sparrow`
 - G. Does not compile.
9. Which of the following legally fill in the blank so you can run the `main()` method from the command line? (Choose all that apply)
- ```
public static void main(_____)
```
- A. `String[] _names`
  - B. `String[] 123`
  - C. `String abc[]`
  - D. `String _Names[]`
  - E. `String... $n`
  - F. `String names`
  - G. None of the above.
10. Which of the following are legal entry point methods that can be run from the command line? (Choose all that apply)
- A. `private static void main(String[] args)`
  - B. `public static final main(String[] args)`
  - C. `public void main(String[] args)`
  - D. `public static void test(String[] args)`
  - E. `public static void main(String[] args)`
  - F. `public static main(String[] args)`
  - G. None of the above.

11. Which of the following are true? (Choose all that apply)
- A. An instance variable of type `double` defaults to `null`.
  - B. An instance variable of type `int` defaults to `null`.
  - C. An instance variable of type `String` defaults to `null`.
  - D. An instance variable of type `double` defaults to `0.0`.
  - E. An instance variable of type `int` defaults to `0.0`.
  - F. An instance variable of type `String` defaults to `0.0`.
  - G. None of the above.
12. Which of the following are true? (Choose all that apply)
- A. A local variable of type `boolean` defaults to `null`.
  - B. A local variable of type `float` defaults to `0`.
  - C. A local variable of type `Object` defaults to `null`.
  - D. A local variable of type `boolean` defaults to `false`.
  - E. A local variable of type `boolean` defaults to `true`.
  - F. A local variable of type `float` defaults to `0.0`.
  - G. None of the above.
13. Which of the following are true? (Choose all that apply)
- A. An instance variable of type `boolean` defaults to `false`.
  - B. An instance variable of type `boolean` defaults to `true`.
  - C. An instance variable of type `boolean` defaults to `null`.
  - D. An instance variable of type `int` defaults to `0`.
  - E. An instance variable of type `int` defaults to `0.0`.
  - F. An instance variable of type `int` defaults to `null`.
  - G. None of the above.
14. Given the following class in the file `/my/directory/named/A/Bird.java`:
- ```
INSERT CODE HERE
public class Bird { }
```
- Which of the following replaces `INSERT CODE HERE` if we compile from `/my/directory/`? (Choose all that apply)
- A. `package my.directory.named.a;`
 - B. `package my.directory.named.A;`
 - C. `package named.a;`
 - D. `package named.A;`
 - E. `package a;`
 - F. `package A;`
 - G. Does not compile.

15. Which of the following lines of code compile? (Choose all that apply)
- A. `int i1 = 1_234;`
 - B. `double d1 = 1_234_.0;`
 - C. `double d2 = 1_234._0;`
 - D. `double d3 = 1_234.0_;`
 - E. `double d4 = 1_234.0;`
 - F. None of the above.
16. Given the following class, which of the following lines of code can replace INSERT CODE HERE to make the code compile? (Choose all that apply)
- ```
public class Price {
 public void admission() {
 INSERT CODE HERE
 System.out.println(amount);
 }
}
```
- A. `int amount = 9L;`
  - B. `int amount = 0b101;`
  - C. `int amount = 0xE;`
  - D. `double amount = 0xE;`
  - E. `double amount = 1_2_.0_0;`
  - F. `int amount = 1_2_;`
  - G. None of the above.
17. Which of the following are true? (Choose all that apply)
- ```
public class Bunny {  
    public static void main(String[] args) {  
        Bunny bun = new Bunny();  
    }  
}
```
- A. Bunny is a class.
 - B. bun is a class.
 - C. main is a class.
 - D. Bunny is a reference to an object.
 - E. bun is a reference to an object.
 - F. main is a reference to an object.
 - G. None of the above.
18. Which represent the order in which the following statements can be assembled into a program that will compile successfully? (Choose all that apply)
- A: `class Rabbit {}`
 - B: `import java.util.*;`
 - C: `package animals;`

- A. A, B, C
- B. B, C, A
- C. C, B, A
- D. B, A
- E. C, A
- F. A, C
- G. A, B

19. Suppose we have a class named `Rabbit`. Which of the following statements are true? (Choose all that apply)

```
1: public class Rabbit {  
2:     public static void main(String[] args) {  
3:         Rabbit one = new Rabbit();  
4:         Rabbit two = new Rabbit();  
5:         Rabbit three = one;  
6:         one = null;  
7:         Rabbit four = one;  
8:         three = null;  
9:         two = null;  
10:        two = new Rabbit();  
11:        System.gc();  
12:    } }
```

- A. The `Rabbit` object from line 3 is first eligible for garbage collection immediately following line 6.
- B. The `Rabbit` object from line 3 is first eligible for garbage collection immediately following line 8.
- C. The `Rabbit` object from line 3 is first eligible for garbage collection immediately following line 12.
- D. The `Rabbit` object from line 4 is first eligible for garbage collection immediately following line 9.
- E. The `Rabbit` object from line 4 is first eligible for garbage collection immediately following line 11.
- F. The `Rabbit` object from line 4 is first eligible for garbage collection immediately following line 12.

20. What is true about the following code? (Choose all that apply)

```
public class Bear {  
    protected void finalize() {  
        System.out.println("Roar!");  
    }  
}
```

```
public static void main(String[] args) {  
    Bear bear = new Bear();  
    bear = null;  
    System.gc();  
} }
```

- A. `finalize()` is guaranteed to be called.
- B. `finalize()` might or might not be called.
- C. `finalize()` is guaranteed not to be called.
- D. Garbage collection is guaranteed to run.
- E. Garbage collection might or might not run.
- F. Garbage collection is guaranteed not to run.
- G. The code does not compile.

21. What does the following code output?

```
1: public class Salmon {  
2:     int count;  
3:     public void Salmon() {  
4:         count = 4;  
5:     }  
6: public static void main(String[] args) {  
7:     Salmon s = new Salmon();  
8:     System.out.println(s.count);  
9: } }
```

- A. 0
- B. 4
- C. Compilation fails on line 3.
- D. Compilation fails on line 4.
- E. Compilation fails on line 7.
- F. Compilation fails on line 8.

22. Which of the following are true statements? (Choose all that apply)

- A. Java allows operator overloading.
- B. Java code compiled on Windows can run on Linux.
- C. Java has pointers to specific locations in memory.
- D. Java is a procedural language.
- E. Java is an object-oriented language.
- F. Java is a functional programming language.

- 23.** Which of the following are true? (Choose all that apply)
- A.** javac compiles a .class file into a .java file.
 - B.** javac compiles a .java file into a .bytecode file.
 - C.** javac compiles a .java file into a .class file.
 - D.** Java takes the name of the class as a parameter.
 - E.** Java takes the name of the .bytecode file as a parameter.
 - F.** Java takes the name of the .class file as a parameter.

Understand Java operator precedence. Most Java operators you'll work with are binary, but the number of expressions is often greater than two. Therefore, you must understand the order in which Java will evaluate each operator symbol.

Be able to write code that uses parentheses to override operator precedence. You can use parentheses in your code to manually change the order of precedence.

Understand if and switch decision control statements. The if-then and switch statements come up frequently throughout the exam in questions unrelated to decision control, so make sure you fully understand these basic building blocks of Java.

Understand loop statements. Know the syntactical structure of all loops, including while, do-while, and for. Each loop has its own special properties and structures. Also, be familiar with the enhanced for-each loops that iterate over lists.

Understand how break and continue can change flow control. Know how to change the flow control within a statement by applying a break or continue command. Also know which control statements can accept break statements and which can accept continue statements. Finally, understand how these statements work inside embedded loops or switch statements.

Review Questions

1. Which of the following Java operators can be used with boolean variables? (Choose all that apply)
 - A. `==`
 - B. `+`
 - C. `--`
 - D. `!`
 - E. `%`
 - F. `<=`
2. What data type (or types) will allow the following code snippet to compile? (Choose all that apply)

```
byte x = 5;
byte y = 10;
----- z = x + y;
```

 - A. `int`
 - B. `long`
 - C. `boolean`
 - D. `double`
 - E. `short`
 - F. `byte`
3. What is the output of the following application?

```
1: public class CompareValues {
2:     public static void main(String[] args) {
3:         int x = 0;
4:         while(x++ < 10) {}
5:         String message = x > 10 ? "Greater than" : false;
6:         System.out.println(message+" "+x);
7:     }
8: }
```

 - A. Greater than,10
 - B. false,10
 - C. Greater than,11
 - D. false,11
 - E. The code will not compile because of line 4.
 - F. The code will not compile because of line 5.

4. What change would allow the following code snippet to compile? (Choose all that apply)

```
3: long x = 10;  
4: int y = 2 * x;
```

- A. No change; it compiles as is.
- B. Cast x on line 4 to int.
- C. Change the data type of x on line 3 to short.
- D. Cast 2 * x on line 4 to int.
- E. Change the data type of y on line 4 to short.
- F. Change the data type of y on line 4 to long.

5. What is the output of the following code snippet?

```
3: java.util.List<Integer> list = new java.util.ArrayList<Integer>();  
4: list.add(10);  
5: list.add(14);  
6: for(int x : list) {  
7:     System.out.print(x + ", ");  
8:     break;  
9: }
```

- A. 10, 14,
- B. 10, 14
- C. 10,
- D. The code will not compile because of line 7.
- E. The code will not compile because of line 8.
- F. The code contains an infinite loop and does not terminate.

6. What is the output of the following code snippet?

```
3: int x = 4;  
4: long y = x * 4 - x++;  
5: if(y<10) System.out.println("Too Low");  
6: else System.out.println("Just right");  
7: else System.out.println("Too High");
```

- A. Too Low
- B. Just Right
- C. Too High
- D. Compiles but throws a NullPointerException.
- E. The code will not compile because of line 6.
- F. The code will not compile because of line 7.

7. What is the output of the following code?
- ```
1: public class TernaryTester {
2: public static void main(String[] args) {
3: int x = 5;
4: System.out.println(x > 2 ? x < 4 ? 10 : 8 : 7);
5: }}
```
- A. 5  
B. 4  
C. 10  
D. 8  
E. 7  
F. The code will not compile because of line 4.
8. What is the output of the following code snippet?
- ```
3: boolean x = true, z = true;  
4: int y = 20;  
5: x = (y != 10) ^ (z=false);  
6: System.out.println(x+", "+y+", "+z);
```
- A. true, 10, true
B. true, 20, false
C. false, 20, true
D. false, 20, false
E. false, 20, true
F. The code will not compile because of line 5.
9. How many times will the following code print "Hello World"?
- ```
3: for(int i=0; i<10 ;) {
4: i = i++;
5: System.out.println("Hello World");
6: }
```
- A. 9  
B. 10  
C. 11  
D. The code will not compile because of line 3.  
E. The code will not compile because of line 5.  
F. The code contains an infinite loop and does not terminate.
10. What is the output of the following code?
- ```
3: byte a = 40, b = 50;  
4: byte sum = (byte) a + b;  
5: System.out.println(sum);
```

- A. 40
 - B. 50
 - C. 90
 - D. The code will not compile because of line 4.
 - E. An undefined value.
11. What is the output of the following code?
- ```
1: public class ArithmeticSample {
2: public static void main(String[] args) {
3: int x = 5 * 4 % 3;
4: System.out.println(x);
5: }}
```
- A. 2
  - B. 3
  - C. 5
  - D. 6
  - E. The code will not compile because of line 3.
12. What is the output of the following code snippet?
- ```
3: int x = 0;  
4: String s = null;  
5: if(x == s) System.out.println("Success");  
6: else System.out.println("Failure");
```
- A. Success
 - B. Failure
 - C. The code will not compile because of line 4.
 - D. The code will not compile because of line 5.
13. What is the output of the following code snippet?
- ```
3: int x1 = 50, x2 = 75;
4: boolean b = x1 >= x2;
5: if(b = true) System.out.println("Success");
6: else System.out.println("Failure");
```
- A. Success
  - B. Failure
  - C. The code will not compile because of line 4.
  - D. The code will not compile because of line 5.
14. What is the output of the following code snippet?
- ```
3: int c = 7;  
4: int result = 4;
```

```
5: result += ++c;  
6: System.out.println(result);
```

- A. 8
- B. 11
- C. 12
- D. 15
- E. 16
- F. The code will not compile because of line 5.

15. What is the output of the following code snippet?

```
3: int x = 1, y = 15;  
4: while x < 10  
5:     y--;  
6:     x++;  
7: System.out.println(x+", "+y);
```

- A. 10, 5
- B. 10, 6
- C. 11, 5
- D. The code will not compile because of line 3.
- E. The code will not compile because of line 4.
- F. The code contains an infinite loop and does not terminate.

16. What is the output of the following code snippet?

```
3: do {  
4:     int y = 1;  
5:     System.out.print(y++ + " ");  
6: } while(y <= 10);
```

- A. 1 2 3 4 5 6 7 8 9
- B. 1 2 3 4 5 6 7 8 9 10
- C. 1 2 3 4 5 6 7 8 9 10 11
- D. The code will not compile because of line 6.
- E. The code contains an infinite loop and does not terminate.

17. What is the output of the following code snippet?

```
3: boolean keepGoing = true;  
4: int result = 15, i = 10;  
5: do {  
6:     i--;  
7:     if(i==8) keepGoing = false;  
8:     result -= 2;  
9: } while(keepGoing);
```

```
10: System.out.println(result);
```

- A. 7
- B. 9
- C. 10
- D. 11
- E. 15
- F. The code will not compile because of line 8.

18. What is the output of the following code snippet?

```
3: int count = 0;
4: ROW_LOOP: for(int row = 1; row <=3; row++)
5:   for(int col = 1; col <=2 ; col++) {
6:     if(row * col % 2 == 0) continue ROW_LOOP;
7:     count++;
8:   }
9: System.out.println(count);
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 6
- F. The code will not compile because of line 6.

19. What is the result of the following code snippet?

```
3: int m = 9, n = 1, x = 0;
4: while(m > n) {
5:   m--;
6:   n += 2;
7:   x += m + n;
8: }
9: System.out.println(x);
```

- A. 11
- B. 13
- C. 23
- D. 36
- E. 50
- F. The code will not compile because of line 7.

20. What is the result of the following code snippet?

```
3: final char a = 'A', d = 'D';
4: char grade = 'B';
```

```
5: switch(grade) {  
6:   case a:  
7:     case 'B': System.out.print("great");  
8:     case 'C': System.out.print("good"); break;  
9:     case d:  
10:    case 'F': System.out.print("not good");  
11: }
```

- A.** great
- B.** greatgood
- C.** The code will not compile because of line 3.
- D.** The code will not compile because of line 6.
- E.** The code will not compile because of lines 6 and 9.

Review Questions

1. What is output by the following code? (Choose all that apply)

```
1: public class Fish {  
2:     public static void main(String[] args) {  
3:         int numFish = 4;  
4:         String fishType = "tuna";  
5:         String anotherFish = numFish + 1;  
6:         System.out.println(anotherFish + " " + fishType);  
7:         System.out.println(numFish + " " + 1);  
8:     } }
```

- A. 4 1
- B. 41
- C. 5
- D. 5 tuna
- E. 5tuna
- F. 51tuna
- G. The code does not compile.

2. Which of the following are output by this code? (Choose all that apply)

```
3: String s = "Hello";  
4: String t = new String(s);  
5: if ("Hello".equals(s)) System.out.println("one");  
6: if (t == s) System.out.println("two");  
7: if (t.equals(s)) System.out.println("three");  
8: if ("Hello" == s) System.out.println("four");  
9: if ("Hello" == t) System.out.println("five");
```

- A. one
- B. two
- C. three
- D. four
- E. five
- F. The code does not compile.

3. Which are true statements? (Choose all that apply)

- A. An immutable object can be modified.
- B. An immutable object cannot be modified.
- C. An immutable object can be garbage collected.

- D. An immutable object cannot be garbage collected.
 - E. String is immutable.
 - F. StringBuffer is immutable.
 - G. StringBuilder is immutable.
4. What is the result of the following code?
- ```
7: StringBuilder sb = new StringBuilder();
8: sb.append("aaa").insert(1, "bb").insert(4, "ccc");
9: System.out.println(sb);
```
- A. abbaaccc
  - B. abbaccca
  - C. bbaaaccc
  - D. bbaaccca
  - E. An exception is thrown.
  - F. The code does not compile.
5. What is the result of the following code?
- ```
2: String s1 = "java";  
3: StringBuilder s2 = new StringBuilder("java");  
4: if (s1 == s2)  
5:   System.out.print("1");  
6: if (s1.equals(s2))  
7:   System.out.print("2");
```
- A. 1
 - B. 2
 - C. 12
 - D. No output is printed.
 - E. An exception is thrown.
 - F. The code does not compile.
6. What is the result of the following code?
- ```
public class Lion {
 public void roar(String roar1, StringBuilder roar2) {
 roar1.concat("!!!");
 roar2.append("!!!");
 }
 public static void main(String[] args) {
 String roar1 = "roar";
 StringBuilder roar2 = new StringBuilder("roar");
 new Lion().roar(roar1, roar2);
 }
}
```

```
 System.out.println(roar1 + " " + roar2);
} }
```

- A. roar roar
  - B. roar roar!!!
  - C. roar!!! roar
  - D. roar!!! roar!!!
  - E. An exception is thrown.
  - F. The code does not compile.
7. Which are the results of the following code? (Choose all that apply)
- ```
String letters = "abcdef";  
System.out.println(letters.length());  
System.out.println(letters.charAt(3));  
System.out.println(letters.charAt(6));
```
- A. 5
 - B. 6
 - C. c
 - D. d
 - E. An exception is thrown.
 - F. The code does not compile.
8. Which are the results of the following code? (Choose all that apply)
- ```
String numbers = "012345678";
System.out.println(numbers.substring(1, 3));
System.out.println(numbers.substring(7, 7));
System.out.println(numbers.substring(7));
```
- A. 12
  - B. 123
  - C. 7
  - D. 78
  - E. A blank line.
  - F. An exception is thrown.
  - G. The code does not compile.
9. What is the result of the following code?
- ```
3: String s = "purr";  
4: s.toUpperCase();  
5: s.trim();  
6: s.substring(1, 3);
```

```
7: s += " two";  
8: System.out.println(s.length());
```

- A. 2
- B. 4
- C. 8
- D. 10
- E. An exception is thrown.
- F. The code does not compile.

10. What is the result of the following code? (Choose all that apply)

```
13: String a = "";  
14: a += 2;  
15: a += 'c';  
16: a += false;  
17: if ( a == "2cfalse") System.out.println("==");  
18: if ( a.equals("2cfalse")) System.out.println("equals");
```

- A. Compile error on line 14.
- B. Compile error on line 15.
- C. Compile error on line 16.
- D. Compile error on another line.
- E. ==
- F. equals
- G. An exception is thrown.

11. What is the result of the following code?

```
4: int total = 0;  
5: StringBuilder letters = new StringBuilder("abcdefg");  
6: total += letters.substring(1, 2).length();  
7: total += letters.substring(6, 6).length();  
8: total += letters.substring(6, 5).length();  
9: System.out.println(total);
```

- A. 1
- B. 2
- C. 3
- D. 7
- E. An exception is thrown.
- F. The code does not compile.

12. What is the result of the following code? (Choose all that apply)
- ```
StringBuilder numbers = new StringBuilder("0123456789");
numbers.delete(2, 8);
numbers.append("-").insert(2, "+");
System.out.println(numbers);
```
- A. 01+89-
  - B. 012+9-
  - C. 012+-9
  - D. 0123456789
  - E. An exception is thrown.
  - F. The code does not compile.
13. What is the result of the following code?
- ```
StringBuilder b = "rumble";
b.append(4).deleteCharAt(3).delete(3, b.length() - 1);
System.out.println(b);
```
- A. rum
 - B. rum4
 - C. rumb4
 - D. rumble4
 - E. An exception is thrown.
 - F. The code does not compile.
14. Which of the following can replace line 4 to print "avaJ"? (Choose all that apply)
- ```
3: StringBuilder puzzle = new StringBuilder("Java");
4: // INSERT CODE HERE
5: System.out.println(puzzle);
```
- A. puzzle.reverse();
  - B. puzzle.append("vaJ\$").substring(0, 4);
  - C. puzzle.append("vaJ\$").delete(0, 3).deleteCharAt(puzzle.length() - 1);
  - D. puzzle.append("vaJ\$").delete(0, 3).deleteCharAt(puzzle.length());
  - E. None of the above.
15. Which of these array declarations is not legal? (Choose all that apply)
- A. `int[][] scores = new int[5][];`
  - B. `Object[][][] cubbies = new Object[3][0][5];`
  - C. `String beans[] = new beans[6];`
  - D. `java.util.Date[] dates[] = new java.util.Date[2][];`
  - E. `int[][] types = new int[];`
  - F. `int[][] java = new int[][];`

16. Which of these compile when replacing line 8? (Choose all that apply)

```
7: char[] c = new char[2];
```

```
8: // INSERT CODE HERE
```

- A. `int length = c.capacity;`
- B. `int length = c.capacity();`
- C. `int length = c.length;`
- D. `int length = c.length();`
- E. `int length = c.size;`
- F. `int length = c.size();`
- G. None of the above.

17. Which of these compile when replacing line 8? (Choose all that apply)

```
7: ArrayList l = new ArrayList();
```

```
8: // INSERT CODE HERE
```

- A. `int length = l.capacity;`
- B. `int length = l.capacity();`
- C. `int length = l.length;`
- D. `int length = l.length();`
- E. `int length = l.size;`
- F. `int length = l.size();`
- G. None of the above.

18. Which of the following are true? (Choose all that apply)

- A. An array has a fixed size.
- B. An `ArrayList` has a fixed size.
- C. An array allows multiple dimensions.
- D. An array is ordered.
- E. An `ArrayList` is ordered.
- F. An array is immutable.
- G. An `ArrayList` is immutable.

19. Which of the following are true? (Choose all that apply)

- A. Two arrays with the same content are equal.
- B. Two `ArrayLists` with the same content are equal.
- C. If you call `remove(0)` using an empty `ArrayList` object, it will compile successfully.
- D. If you call `remove(0)` using an empty `ArrayList` object, it will run successfully.
- E. None of the above.

**20.** What is the result of the following statements?

```
6: List<String> list = new ArrayList<String>();
7: list.add("one");
8: list.add("two");
9: list.add(7);
10: for(String s : list) System.out.print(s);
```

- A. onetwo
- B. onetwo7
- C. onetwo followed by an exception
- D. Compiler error on line 9.
- E. Compiler error on line 10.

**21.** What is the result of the following statements?

```
3: ArrayList<Integer> values = new ArrayList<>();
4: values.add(4);
5: values.add(5);
6: values.set(1, 6);
7: values.remove(0);
8: for (Integer v : values) System.out.print(v);
```

- A. 4
- B. 5
- C. 6
- D. 46
- E. 45
- F. An exception is thrown.
- G. The code does not compile.

**22.** What is the result of the following?

```
int[] random = { 6, -4, 12, 0, -10 };
int x = 12;
int y = Arrays.binarySearch(random, x);
System.out.println(y);
```

- A. 2
- B. 4
- C. 6
- D. The result is undefined.
- E. An exception is thrown.
- F. The code does not compile.

**23.** What is the result of the following?

```
4: List<Integer> list = Arrays.asList(10, 4, -1, 5);
5: Collections.sort(list);
6: Integer array[] = list.toArray(new Integer[4]);
7: System.out.println(array[0]);
```

- A. -1
- B. 10
- C. Compiler error on line 4.
- D. Compiler error on line 5.
- E. Compiler error on line 6.
- F. An exception is thrown.

**24.** What is the result of the following?

```
6: String [] names = {"Tom", "Dick", "Harry"};
7: List<String> list = names.asList();
8: list.set(0, "Sue");
9: System.out.println(names[0]);
```

- A. Sue
- B. Tom
- C. Compiler error on line 7.
- D. Compiler error on line 8.
- E. An exception is thrown.

**25.** What is the result of the following?

```
List<String> hex = Arrays.asList("30", "8", "3A", "FF");
Collections.sort(hex);
int x = Collections.binarySearch(hex, "8");
int y = Collections.binarySearch(hex, "3A");
int z = Collections.binarySearch(hex, "4F");
System.out.println(x + " " + y + " " + z);
```

- A. 0 1 -2
- B. 0 1 -3
- C. 2 1 -2
- D. 2 1 -3
- E. None of the above.
- F. The code doesn't compile.

**26.** Which of the following are true statements about the following code? (Choose all that apply)

```
4: List<Integer> ages = new ArrayList<>();
5: ages.add(Integer.parseInt("5"));
```

```
6: ages.add(Integer.valueOf("6"));
7: ages.add(7);
8: ages.add(null);
9: for (int age : ages) System.out.print(age);
```

- A. The code compiles.
- B. The code throws a runtime exception.
- C. Exactly one of the add statements uses autoboxing.
- D. Exactly two of the add statements use autoboxing.
- E. Exactly three of the add statements use autoboxing.

27. What is the result of the following?

```
List<String> one = new ArrayList<String>();
one.add("abc");
List<String> two = new ArrayList<>();
two.add("abc");
if (one == two)
 System.out.println("A");
else if (one.equals(two))
 System.out.println("B");
else
 System.out.println("C");
```

- A. A
- B. B
- C. C
- D. An exception is thrown.
- E. The code does not compile.

28. Which of the following can be inserted into the blank to create a date of June 21, 2014?  
(Choose all that apply)

```
import java.time.*;
```

```
public class StartOfSummer {
```

```
 public static void main(String[] args) {
 LocalDate date = -----
 }
```

```
}
```

- A. new LocalDate(2014, 5, 21);
- B. new LocalDate(2014, 6, 21);
- C. LocalDate.of(2014, 5, 21);

- D. `LocalDate.of(2014, 6, 21);`
- E. `LocalDate.of(2014, Calendar.JUNE, 21);`
- F. `LocalDate.of(2014, Month.JUNE, 21);`

**29.** What is the output of the following code?

```
LocalDate date = LocalDate.parse("2018-04-30", DateTimeFormatter.ISO_LOCAL_
DATE);
date.plusDays(2);
date.plusHours(3);
System.out.println(date.getYear() + " " + date.getMonth() + " "
+ date.getDayOfMonth());
```

- A. 2018 APRIL 2
- B. 2018 APRIL 30
- C. 2018 MAY 2
- D. The code does not compile.
- E. A runtime exception is thrown.

**30.** What is the output of the following code?

```
LocalDate date = LocalDate.of(2018, Month.APRIL, 40);
System.out.println(date.getYear() + " " + date.getMonth() + " "
+ date.getDayOfMonth());
```

- A. 2018 APRIL 4
- B. 2018 APRIL 30
- C. 2018 MAY 10
- D. Another date.
- E. The code does not compile.
- F. A runtime exception is thrown.

**31.** What is the output of the following code?

```
LocalDate date = LocalDate.of(2018, Month.APRIL, 30);
date.plusDays(2);
date.plusYears(3);
System.out.println(date.getYear() + " " + date.getMonth() + " "
+ date.getDayOfMonth());
```

- A. 2018 APRIL 2
- B. 2018 APRIL 30
- C. 2018 MAY 2
- D. 2021 APRIL 2

- E. 2021 APRIL 30
- F. 2021 MAY 2
- G. A runtime exception is thrown.

**32.** What is the output of the following code?

```
LocalDateTime d = LocalDateTime.of(2015, 5, 10, 11, 22, 33);
Period p = Period.of(1, 2, 3);
d = d.minus(p);
DateTimeFormatter f = DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT);
System.out.println(d.format(f));
```

- A. 3/7/14 11:22 AM
- B. 5/10/15 11:22 AM
- C. 3/7/14
- D. 5/10/15
- E. 11:22 AM
- F. The code does not compile.
- G. A runtime exception is thrown.

**33.** What is the output of the following code?

```
LocalDateTime d = LocalDateTime.of(2015, 5, 10, 11, 22, 33);
Period p = Period.ofDays(1).ofYears(2);
d = d.minus(p);
DateTimeFormatter f = DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);
System.out.println(f.format(d));
```

- A. 5/9/13 11:22 AM
- B. 5/10/13 11:22 AM
- C. 5/9/14
- D. 5/10/14
- E. The code does not compile.
- F. A runtime exception is thrown.



## Review Questions

1. Which of the following can fill in the blank in this code to make it compile? (Choose all that apply)

```
public class Ant {
 _____ void method() { }
}
```

- A. default
  - B. final
  - C. private
  - D. Public
  - E. String
  - F. zzz:
2. Which of the following compile? (Choose all that apply)
- A. `final static void method4() { }`
  - B. `public final int void method() { }`
  - C. `private void int method() { }`
  - D. `static final void method3() { }`
  - E. `void final method() {}`
  - F. `void public method() { }`
3. Which of the following methods compile? (Choose all that apply)
- A. `public void methodA() { return;}`
  - B. `public void methodB() { return null;}`
  - C. `public void methodD() {}`
  - D. `public int methodD() { return 9;}`
  - E. `public int methodE() { return 9.0;}`
  - F. `public int methodF() { return;}`
  - G. `public int methodG() { return null;}`
4. Which of the following compile? (Choose all that apply)
- A. `public void moreA(int... nums) {}`
  - B. `public void moreB(String values, int... nums) {}`
  - C. `public void moreC(int... nums, String values) {}`
  - D. `public void moreD(String... values, int... nums) {}`
  - E. `public void moreE(String[] values, ...int nums) {}`
  - F. `public void moreF(String... values, int[] nums) {}`
  - G. `public void moreG(String[] values, int[] nums) {}`

5. Given the following method, which of the method calls return 2? (Choose all that apply)

```
public int howMany(boolean b, boolean... b2) {
 return b2.length;
}
```

- A. `howMany();`
  - B. `howMany(true);`
  - C. `howMany(true, true);`
  - D. `howMany(true, true, true);`
  - E. `howMany(true, {true});`
  - F. `howMany(true, {true, true});`
  - G. `howMany(true, new boolean[2]);`
6. Which of the following are true? (Choose all that apply)
- A. Package private access is more lenient than protected access.
  - B. A public class that has private fields and package private methods is not visible to classes outside the package.
  - C. You can use access modifiers so only some of the classes in a package see a particular package private class.
  - D. You can use access modifiers to allow read access to all methods, but not any instance variables.
  - E. You can use access modifiers to restrict read access to all classes that begin with the word `Test`.
7. Given the following `my.school.ClassRoom` and `my.city.School` class definitions, which line numbers in `main()` generate a compiler error? (Choose all that apply)

```
1: package my.school;
2: public class Classroom {
3: private int roomNumber;
4: protected String teacherName;
5: static int globalKey = 54321;
6: public int floor = 3;
7: Classroom(int r, String t) {
8: roomNumber = r;
9: teacherName = t; } }
```

```
1: package my.city;
2: import my.school.*;
3: public class School {
4: public static void main(String[] args) {
5: System.out.println(Classroom.globalKey);
6: Classroom room = new Classroom(101, ""Mrs. Anderson");
```

```
7: System.out.println(room.roomNumber);
8: System.out.println(room.floor);
9: System.out.println(room.teacherName); } }
```

- A. None, the code compiles fine.
  - B. Line 5
  - C. Line 6
  - D. Line 7
  - E. Line 8
  - F. Line 9
8. Which of the following are true? (Choose all that apply)
- A. Encapsulation uses package private instance variables.
  - B. Encapsulation uses private instance variables.
  - C. Encapsulation allows setters.
  - D. Immutability uses package private instance variables.
  - E. Immutability uses private instance variables.
  - F. Immutability allows setters.
9. Which are methods using JavaBeans naming conventions for accessors and mutators? (Choose all that apply)
- A. `public boolean getCanSwim() { return canSwim;}`
  - B. `public boolean canSwim() { return numberWings;}`
  - C. `public int getNumWings() { return numberWings;}`
  - D. `public int numWings() { return numberWings;}`
  - E. `public void setCanSwim(boolean b) { canSwim = b;}`
10. What is the output of the following code?
- ```
1: package rope;
2: public class Rope {
3:     public static int LENGTH = 5;
4:     static {
5:         LENGTH = 10;
6:     }
```

```
7: public static void swing() {  
8:     System.out.print("swing ");  
9: }  
10: }
```

```
1: import rope.*;  
2: import static rope.Rope.*;  
3: public class Chimp {  
4:     public static void main(String[] args) {  
5:         Rope.swing();  
6:         new Rope().swing();  
7:         System.out.println(LENGTH);  
8:     }  
9: }
```

- A. swing swing 5
- B. swing swing 10
- C. Compiler error on line 2 of Chimp.
- D. Compiler error on line 5 of Chimp.
- E. Compiler error on line 6 of Chimp.
- F. Compiler error on line 7 of Chimp.

11. Which are true of the following code? (Choose all that apply)

```
1: public class Rope {  
2:     public static void swing() {  
3:         System.out.print("swing ");  
4:     }  
5:     public void climb() {  
6:         System.out.println("climb ");  
7:     }  
8:     public static void play() {  
9:         swing();  
10:        climb();  
11:    }  
12:    public static void main(String[] args) {  
13:        Rope rope = new Rope();  
14:        rope.play();  
15:        Rope rope2 = null;  
16:        rope2.play();  
17:    }  
18: }
```

- A. The code compiles as is.
- B. There is exactly one compiler error in the code.
- C. There are exactly two compiler errors in the code.
- D. If the lines with compiler errors are removed, the output is `climb climb`.
- E. If the lines with compiler errors are removed, the output is `swing swing`.
- F. If the lines with compile errors are removed, the code throws a `NullPointerException`.

12. What is the output of the following code?

```
import rope.*;
import static rope.Rope.*;
public class RopeSwing {
    private static Rope rope1 = new Rope();
    private static Rope rope2 = new Rope();
    {
        System.out.println(rope1.length);
    }
    public static void main(String[] args) {
        rope1.length = 2;
        rope2.length = 8;
        System.out.println(rope1.length);
    }
}
```

```
package rope;
public class Rope {
    public static int length = 0;
}
```

- A. 02
- B. 08
- C. 2
- D. 8
- E. The code does not compile.
- F. An exception is thrown.

13. How many compiler errors are in the following code?

```
1: public class RopeSwing {
2:     private static final String leftRope;
3:     private static final String rightRope;
4:     private static final String bench;
5:     private static final String name = "name";
```

```
6:  static {
7:      leftRope = "left";
8:      rightRope = "right";
9:  }
10: static {
11:     name = "name";
12:     rightRope = "right";
13: }
14: public static void main(String[] args) {
15:     bench = "bench";
16: }
17: }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4
- F. 5

14. Which of the following can replace line 2 to make this code compile? (Choose all that apply)

```
1: import java.util.*;
2: // INSERT CODE HERE
3: public class Imports {
4:     public void method(ArrayList<String> list) {
5:         sort(list);
6:     }
7: }
```

- A. `import static java.util.Collections;`
- B. `import static java.util.Collections.*;`
- C. `import static java.util.Collections.sort(ArrayList<String>);`
- D. `static import java.util.Collections;`
- E. `static import java.util.Collections.*;`
- F. `static import java.util.Collections.sort(ArrayList<String>);`

15. What is the result of the following statements?

```
1: public class Test {
2:     public void print(byte x) {
3:         System.out.print("byte");
4:     }
5:     public void print(int x) {
6:         System.out.print("int");
7:     }
8: }
```

```
7:    }
8:    public void print(float x) {
9:        System.out.print("float");
10:    }
11:    public void print(Object x) {
12:        System.out.print("Object");
13:    }
14:    public static void main(String[] args) {
15:        Test t = new Test();
16:        short s = 123;
17:        t.print(s);
18:        t.print(true);
19:        t.print(6.789);
20:    }
21: }
```

- A. byteFloatObject
- B. intFloatObject
- C. byteObjectFloat
- D. intObjectFloat
- E. intObjectObject
- F. byteObjectObject

16. What is the result of the following program?

```
1: public class Squares {
2:     public static long square(int x) {
3:         long y = x * (long) x;
4:         x = -1;
5:         return y;
6:     }
7:     public static void main(String[] args) {
8:         int value = 9;
9:         long result = square(value);
10:        System.out.println(value);
11:    } }
```

- A. -1
- B. 9
- C. 81
- D. Compiler error on line 9.
- E. Compiler error on a different line.

17. Which of the following are output by the following code? (Choose all that apply)

```
public class StringBuilders {  
    public static StringBuilder work(StringBuilder a,  
    StringBuilder b) {  
        a = new StringBuilder("a");  
        b.append("b");  
        return a;  
    }  
    public static void main(String[] args) {  
        StringBuilder s1 = new StringBuilder("s1");  
        StringBuilder s2 = new StringBuilder("s2");  
        StringBuilder s3 = work(s1, s2);  
        System.out.println("s1 = " + s1);  
        System.out.println("s2 = " + s2);  
        System.out.println("s3 = " + s3);  
    }  
}
```

- A. s1 = a
 - B. s1 = s1
 - C. s2 = s2
 - D. s2 = s2b
 - E. s3 = a
 - F. s3 = null
 - G. The code does not compile.
18. Which of the following are true? (Choose 2)
- A. this() can be called from anywhere in a constructor.
 - B. this() can be called from any instance method in the class.
 - C. this.variableName can be called from any instance method in the class.
 - D. this.variableName can be called from any static method in the class.
 - E. You must include a default constructor in the code if the compiler does not include one.
 - F. You can call the default constructor written by the compiler using this().
 - G. You can access a private constructor with the main() method.
19. Which of these classes compile and use a default constructor? (Choose all that apply)
- A. public class Bird { }
 - B. public class Bird { public bird() {} }
 - C. public class Bird { public bird(String name) {} }
 - D. public class Bird { public Bird() {} }

- E. `public class Bird { Bird(String name) {} }`
- F. `public class Bird { private Bird(int age) {} }`
- G. `public class Bird { void Bird() { } }`

20. Which code can be inserted to have the code print 2?

```
public class BirdSeed {
    private int numberBags;
    boolean call;

    public BirdSeed() {
        // LINE 1
        call = false;
        // LINE 2
    }
    public BirdSeed(int numberBags) {
        this.numberBags = numberBags;
    }
    public static void main(String[] args) {
        BirdSeed seed = new BirdSeed();
        System.out.println(seed.numberBags);
    } }
```

- A. Replace line 1 with `BirdSeed(2);`
- B. Replace line 2 with `BirdSeed(2);`
- C. Replace line 1 with `new BirdSeed(2);`
- D. Replace line 2 with `new BirdSeed(2);`
- E. Replace line 1 with `this(2);`
- F. Replace line 2 with `this(2);`

21. Which of the following complete the constructor so that this code prints out 50? (Choose all that apply)

```
public class Cheetah {
    int numSpots;
    public Cheetah(int numSpots) {
        // INSERT CODE HERE
    }
    public static void main(String[] args) {
        System.out.println(new Cheetah(50).numSpots);
    }
}
```

- A. numSpots = numSpots;
- B. numSpots = this.numSpots;
- C. this.numSpots = numSpots;
- D. numSpots = super.numSpots;
- E. super.numSpots = numSpots;
- F. None of the above.

22. What is the result of the following?

```
1: public class Order {  
2:     static String result = "";  
3:     { result += "c"; }  
4:     static  
5:     { result += "u"; }  
6:     { result += "r"; }  
7: }
```

```
1: public class OrderDriver {  
2:     public static void main(String[] args) {  
3:         System.out.print(Order.result + " ");  
4:         System.out.print(Order.result + " ");  
5:         new Order();  
6:         new Order();  
7:         System.out.print(Order.result + " ");  
8:     }  
9: }
```

- A. curur
- B. ucr cr
- C. u ucr cr
- D. u u cur cur
- E. u u ucr cr
- F. ur ur urc
- G. The code does not compile.

23. What is the result of the following?

```
1: public class Order {  
2:     String value = "t";  
3:     { value += "a"; }  
4:     { value += "c"; }  
5:     public Order() {
```

```
6:     value += "b";
7: }
8: public Order(String s) {
9:     value += s;
10: }
11: public static void main(String[] args) {
12:     Order order = new Order("f");
13:     order = new Order();
14:     System.out.println(order.value);
15: } }
```

- A. tacb
- B. tacf
- C. tacbf
- D. tacfb
- E. tacftacb
- F. The code does not compile.
- G. An exception is thrown.

24. Which of the following will compile when inserted in the following code? (Choose all that apply)

```
public class Order3 {
    final String value1 = "1";
    static String value2 = "2";
    String value3 = "3";
    {
        // CODE SNIPPET 1
    }
    static {
        // CODE SNIPPET 2
    }
}
```

- A. value1 = "d"; instead of // CODE SNIPPET 1
- B. value2 = "e"; instead of // CODE SNIPPET 1
- C. value3 = "f"; instead of // CODE SNIPPET 1
- D. value1 = "g"; instead of // CODE SNIPPET 2
- E. value2 = "h"; instead of // CODE SNIPPET 2
- F. value3 = "i"; instead of // CODE SNIPPET 2

25. Which of the following are true about the following code? (Choose all that apply)

```
public class Create {
    Create() {
        System.out.print("1 ");
    }
    Create(int num) {
        System.out.print("2 ");
    }
    Create(Integer num) {
        System.out.print("3 ");
    }
    Create(Object num) {
        System.out.print("4 ");
    }
    Create(int... nums) {
        System.out.print("5 ");
    }
    public static void main(String[] args) {
        new Create(100);
        new Create(1000L);
    }
}
```

- A. The code prints out 2 4.
 - B. The code prints out 3 4.
 - C. The code prints out 4 2.
 - D. The code prints out 4 4.
 - E. The code prints 3 4 if you remove the constructor `Create(int num)`.
 - F. The code prints 4 4 if you remove the constructor `Create(int num)`.
 - G. The code prints 5 4 if you remove the constructor `Create(int num)`.
26. What is the result of the following class?
- ```
1: import java.util.function.*;
2:
3: public class Panda {
4: int age;
5: public static void main(String[] args) {
6: Panda p1 = new Panda();
7: p1.age = 1;
8: check(p1, p -> p.age < 5);
```

```
9: }
10: private static void check(Panda panda, Predicate<Panda> pred) {
11: String result = pred.test(panda) ? "match" : "not match";
12: System.out.print(result);
13: } }
```

- A. match
- B. not match
- C. Compiler error on line 8.
- D. Compiler error on line 10.
- E. Compiler error on line 11.
- F. A runtime exception is thrown.

27. What is the result of the following code?

```
1: interface Climb {
2: boolean isTooHigh(int height, int limit);
3: }
4:
5: public class Climber {
6: public static void main(String[] args) {
7: check((h, l) -> h.append(l).isEmpty(), 5);
8: }
9: private static void check(Climb climb, int height) {
10: if (climb.isTooHigh(height, 10))
11: System.out.println("too high");
12: else
13: System.out.println("ok");
14: }
15: }
```

- A. ok
- B. too high
- C. Compiler error on line 7.
- D. Compiler error on line 10.
- E. Compiler error on a different line.
- F. A runtime exception is thrown.

28. Which of the following lambda expressions can fill in the blank? (Choose all that apply)

```
List<String> list = new ArrayList<>();
list.removeIf(_____);
```

- A. `s -> s.isEmpty()`
- B. `s -> {s.isEmpty()}`
- C. `s -> {s.isEmpty();}`
- D. `s -> {return s.isEmpty();}`
- E. `String s -> s.isEmpty()`
- F. `(String s) -> s.isEmpty()`

29. Which lambda can replace the `MySecret` class to return the same value? (Choose all that apply)

```
interface Secret {
 String magic(double d);
}
```

```
class MySecret implements Secret {
 public String magic(double d) {
 return "Poof";
 }
}
```

- A. `caller((e) -> "Poof");`
- B. `caller((e) -> {"Poof"});`
- C. `caller((e) -> { String e = ""; "Poof" });`
- D. `caller((e) -> { String e = ""; return "Poof"; });`
- E. `caller((e) -> { String e = ""; return "Poof" });`
- F. `caller((e) -> { String f = ""; return "Poof"; });`

# Review Questions

1. What modifiers are implicitly applied to all interface methods? (Choose all that apply)

- A. protected
- B. public
- C. static
- D. void
- E. abstract
- F. default

2. What is the output of the following code?

```
1: class Mammal {
2: public Mammal(int age) {
3: System.out.print("Mammal");
4: }
5: }
6: public class Platypus extends Mammal {
7: public Platypus() {
8: System.out.print("Platypus");
9: }
10: public static void main(String[] args) {
11: new Mammal(5);
12: }
13: }
```

- A. Platypus
- B. Mammal
- C. PlatypusMammal
- D. MammalPlatypus
- E. The code will not compile because of line 8.
- F. The code will not compile because of line 11.

3. Which of the following statements can be inserted in the blank line so that the code will compile successfully? (Choose all that apply)

```
public interface CanHop {}
public class Frog implements CanHop {
 public static void main(String[] args) {
 _____ frog = new TurtleFrog();
 }
}
```

```
public class BrazilianHornedFrog extends Frog {}
public class TurtleFrog extends Frog {}
```

- A. Frog
- B. TurtleFrog
- C. BrazilianHornedFrog
- D. CanHop
- E. Object
- F. Long

4. Which statement(s) are correct about the following code? (Choose all that apply)

```
public class Rodent {
 protected static Integer chew() throws Exception {
 System.out.println("Rodent is chewing");
 return 1;
 }
}

public class Beaver extends Rodent {
 public Number chew() throws RuntimeException {
 System.out.println("Beaver is chewing on wood");
 return 2;
 }
}
```

- A. It will compile without issue.
  - B. It fails to compile because the type of the exception the method throws is a subclass of the type of exception the parent method throws.
  - C. It fails to compile because the return types are not covariant.
  - D. It fails to compile because the method is protected in the parent class and public in the subclass.
  - E. It fails to compile because of a static modifier mismatch between the two methods.
5. Which of the following may only be hidden and not overridden? (Choose all that apply)
- A. private instance methods
  - B. protected instance methods
  - C. public instance methods
  - D. static methods
  - E. public variables
  - F. private variables

6. Choose the correct statement about the following code:

```
1: interface HasExoskeleton {
2: abstract int getNumberOfSections();
3: }
4: abstract class Insect implements HasExoskeleton {
5: abstract int getNumberOfLegs();
6: }
7: public class Beetle extends Insect {
8: int getNumberOfLegs() { return 6; }
9: }
```

- A. It compiles and runs without issue.
  - B. The code will not compile because of line 2.
  - C. The code will not compile because of line 4.
  - D. The code will not compile because of line 7.
  - E. It compiles but throws an exception at runtime.
7. Which of the following statements about polymorphism are true? (Choose all that apply)
- A. A reference to an object may be cast to a subclass of the object without an explicit cast.
  - B. If a method takes a superclass of three objects, then any of those classes may be passed as a parameter to the method.
  - C. A method that takes a parameter with type `java.lang.Object` will take any reference.
  - D. All cast exceptions can be detected at compile-time.
  - E. By defining a public instance method in the superclass, you guarantee that the specific method will be called in the parent class at runtime.

8. Choose the correct statement about the following code:

```
1: public interface Herbivore {
2: int amount = 10;
3: public static void eatGrass();
4: public int chew() {
5: return 13;
6: }
7: }
```

- A. It compiles and runs without issue.
- B. The code will not compile because of line 2.
- C. The code will not compile because of line 3.
- D. The code will not compile because of line 4.
- E. The code will not compile because of lines 2 and 3.
- F. The code will not compile because of lines 3 and 4.

9. Choose the correct statement about the following code:
- ```
1: public interface CanFly {  
2:     void fly();  
3: }  
4: interface HasWings {  
5:     public abstract Object getWindSpan();  
6: }  
7: abstract class Falcon implements CanFly, HasWings {  
8: }
```
- A. It compiles without issue.
 - B. The code will not compile because of line 2.
 - C. The code will not compile because of line 4.
 - D. The code will not compile because of line 5.
 - E. The code will not compile because of lines 2 and 5.
 - F. The code will not compile because the class `Falcon` doesn't implement the interface methods.
10. Which statements are true for both abstract classes and interfaces? (Choose all that apply)
- A. All methods within them are assumed to be abstract.
 - B. Both can contain `public static final` variables.
 - C. Both can be extended using the `extend` keyword.
 - D. Both can contain default methods.
 - E. Both can contain static methods.
 - F. Neither can be instantiated directly.
 - G. Both inherit `java.lang.Object`.
11. What modifiers are assumed for all interface variables? (Choose all that apply)
- A. `public`
 - B. `protected`
 - C. `private`
 - D. `static`
 - E. `final`
 - F. `abstract`
12. What is the output of the following code?
- ```
1: interface Nocturnal {
2: default boolean isBlind() { return true; }
3: }
4: public class Owl implements Nocturnal {
```

```
5: public boolean isBlind() { return false; }
6: public static void main(String[] args) {
7: Nocturnal nocturnal = (Nocturnal)new Owl();
8: System.out.println(nocturnal.isBlind());
9: }
10: }
```

- A. true
- B. false
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.

**13.** What is the output of the following code?

```
1: class Arthropod
2: public void printName(double input) { System.out
 .print("Arthropod"); }
3: }
4: public class Spider extends Arthropod {
5: public void printName(int input) { System.out.print("Spider"); }
6: public static void main(String[] args) {
7: Spider spider = new Spider();
8: spider.printName(4);
9: spider.printName(9.0);
10: }
11: }
```

- A. SpiderArthropod
- B. ArthropodSpider
- C. SpiderSpider
- D. ArthropodArthropod
- E. The code will not compile because of line 5.
- F. The code will not compile because of line 9.

**14.** Which statements are true about the following code? (Choose all that apply)

```
1: interface HasVocalCords {
2: public abstract void makeSound();
3: }
4: public interface CanBark extends HasVocalCords {
5: public void bark();
6: }
```

- A. The CanBark interface doesn't compile.
  - B. A class that implements HasVocalCords must override the makeSound() method.
  - C. A class that implements CanBark inherits both the makeSound() and bark() methods.
  - D. A class that implements CanBark only inherits the bark() method.
  - E. An interface cannot extend another interface.
15. Which of the following is true about a concrete subclass? (Choose all that apply)
- A. A concrete subclass can be declared as abstract.
  - B. A concrete subclass must implement all inherited abstract methods.
  - C. A concrete subclass must implement all methods defined in an inherited interface.
  - D. A concrete subclass cannot be marked as final.
  - E. Abstract methods cannot be overridden by a concrete subclass.
16. What is the output of the following code?
- ```
1: abstract class Reptile {  
2:     public final void layEggs() { System.out.println("Reptile laying eggs");  
    }  
3:     public static void main(String[] args) {  
4:         Reptile reptile = new Lizard();  
5:         reptile.layEggs();  
6:     }  
7: }  
8: public class Lizard extends Reptile {  
9:     public void layEggs() { System.out.println("Lizard laying eggs"); }  
10: }
```
- A. Reptile laying eggs
 - B. Lizard laying eggs
 - C. The code will not compile because of line 4.
 - D. The code will not compile because of line 5.
 - E. The code will not compile because of line 9.
17. What is the output of the following code?
- ```
1: public abstract class Whale {
2: public abstract void dive() {};
3: public static void main(String[] args) {
4: Whale whale = new Orca();
5: whale.dive();
6: }
7: }
```

```
8: class Orca extends Whale {
9: public void dive(int depth) { System.out.println("Orca diving"); }
10: }
```

- A. Orca diving
- B. The code will not compile because of line 2.
- C. The code will not compile because of line 8.
- D. The code will not compile because of line 9.
- E. The output cannot be determined from the code provided.

18. What is the output of the following code? (Choose all that apply)

```
1: interface Aquatic {
2: public default int getNumberOfGills(int input) { return 2; }
3: }
4: public class ClownFish implements Aquatic {
5: public String getNumberOfGills() { return "4"; }
6: public String getNumberOfGills(int input) { return "6"; }
7: public static void main(String[] args) {
8: System.out.println(new ClownFish().getNumberOfGills(-1));
9: }
10: }
```

- A. 2
- B. 4
- C. 6
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 6.
- F. The code will not compile because of line 8.

19. Which of the following statements can be inserted in the blank so that the code will compile successfully? (Choose all that apply)

```
public class Snake {}
public class Cobra extends Snake {}
public class GardenSnake {}
public class SnakeHandler {
 private Snake snake;
 public void setSnake(Snake snake) { this.snake = snake; }
 public static void main(String[] args) {
 new SnakeHandler().setSnake(_____);
 }
}
```

- A. `new Cobra()`
- B. `new GardenSnake()`
- C. `new Snake()`
- D. `new Object()`
- E. `new String("Snake")`
- F. `null`

20. What is the result of the following code?

```
1: public abstract class Bird {
2: private void fly() { System.out.println("Bird is flying"); }
3: public static void main(String[] args) {
4: Bird bird = new Pelican();
5: bird.fly();
6: }
7: }
8: class Pelican extends Bird {
9: protected void fly() { System.out.println("Pelican is flying"); }
10: }
```

- A. Bird is flying
- B. Pelican is flying
- C. The code will not compile because of line 4.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 9.

# Review Questions

1. Which of the following statements are true? (Choose all that apply)
  - A. Runtime exceptions are the same thing as checked exceptions.
  - B. Runtime exceptions are the same thing as unchecked exceptions.
  - C. You can declare only checked exceptions.
  - D. You can declare only unchecked exceptions.
  - E. You can handle only `Exception` subclasses.
2. Which of the following pairs fill in the blanks to make this code compile? (Choose all that apply)

```
7: public void ohNo() _____ Exception {
8: _____ Exception();
9: }
```

  - A. On line 7, fill in `throw`
  - B. On line 7, fill in `throws`
  - C. On line 8, fill in `throw`
  - D. On line 8, fill in `throw new`
  - E. On line 8, fill in `throws`
  - F. On line 8, fill in `throws new`
3. When are you required to use a `finally` block in a regular `try` statement (not a `try-with-resources`)?
  - A. Never.
  - B. When the program code doesn't terminate on its own.
  - C. When there are no `catch` blocks in a `try` statement.
  - D. When there is exactly one `catch` block in a `try` statement.
  - E. When there are two or more `catch` blocks in a `try` statement.
4. Which exception will the following throw?

```
Object obj = new Integer(3);
String str = (String) obj;
System.out.println(str);
```

  - A. `ArrayIndexOutOfBoundsException`
  - B. `ClassCastException`
  - C. `IllegalArgumentException`
  - D. `NumberFormatException`
  - E. None of the above.

5. Which of the following exceptions are thrown by the JVM? (Choose all that apply)
- A. `ArrayIndexOutOfBoundsException`
  - B. `ExceptionInInitializerError`
  - C. `java.io.IOException`
  - D. `NullPointerException`
  - E. `NumberFormatException`
6. What will happen if you add the statement `System.out.println(5 / 0);` to a working `main()` method?
- A. It will not compile.
  - B. It will not run.
  - C. It will run and throw an `ArithmeticException`.
  - D. It will run and throw an `IllegalArgumentException`.
  - E. None of the above.
7. What is printed besides the stack trace caused by the `NullPointerException` from line 16?

```
1: public class DoSomething {
2: public void go() {
3: System.out.print("A");
4: try {
5: stop();
6: } catch (ArithmeticException e) {
7: System.out.print("B");
8: } finally {
9: System.out.print("C");
10: }
11: System.out.print("D");
12: }
13: public void stop() {
14: System.out.print("E");
15: Object x = null;
16: x.toString();
17: System.out.print("F");
18: }
19: public static void main(String[] args) {
20: new DoSomething().go();
21: }
22: }
```

- A. AE
- B. AEBCD

- C. AEC
  - D. AECD
  - E. No output appears other than the stack trace.
8. What is the output of the following snippet, assuming a and b are both 0?
- ```
3:     try {
4:         return a / b;
5:     } catch (RuntimeException e) {
6:         return -1;
7:     } catch (ArithmeticException e) {
8:         return 0;
9:     } finally {
10:        System.out.print("done");
11:    }
```
- A. -1
 - B. 0
 - C. done-1
 - D. done0
 - E. The code does not compile.
 - F. An uncaught exception is thrown.
9. What is the output of the following program?
- ```
1: public class Laptop {
2: public void start() {
3: try {
4: System.out.print("Starting up ");
5: throw new Exception();
6: } catch (Exception e) {
7: System.out.print("Problem ");
8: System.exit(0);
9: } finally {
10: System.out.print("Shutting down ");
11: }
12: }
13: public static void main(String[] args) {
14: new Laptop().start();
15: } }
```
- A. Starting up
  - B. Starting up Problem
  - C. Starting up Problem Shutting down

- D. Starting up Shutting down
- E. The code does not compile.
- F. An uncaught exception is thrown.

10. What is the output of the following program?

```
1: public class Dog {
2: public String name;
3: public void parseName() {
4: System.out.print("1");
5: try {
6: System.out.print("2");
7: int x = Integer.parseInt(name);
8: System.out.print("3");
9: } catch (NumberFormatException e) {
10: System.out.print("4");
11: }
12: }
13: public static void main(String[] args) {
14: Dog leroy = new Dog();
15: leroy.name = "Leroy";
16: leroy.parseName();
17: System.out.print("5");
18: } }
```

- A. 12
- B. 1234
- C. 1235
- D. 124
- E. 1245
- F. The code does not compile.
- G. An uncaught exception is thrown.

11. What is the output of the following program?

```
1: public class Cat {
2: public String name;
3: public void parseName() {
4: System.out.print("1");
5: try {
6: System.out.print("2");
7: int x = Integer.parseInt(name);
8: System.out.print("3");
```

```
9: } catch (NullPointerException e) {
10: System.out.print("4");
11: }
12: System.out.print("5");
13: }
14: public static void main(String[] args) {
15: Cat leo = new Cat();
16: leo.name = "Leo";
17: leo.parseName();
18: System.out.print("6");
19: }
20: }
```

- A. 12, followed by a stack trace for a `NumberFormatException`
- B. 124, followed by a stack trace for a `NumberFormatException`
- C. 12456
- D. 12456
- E. 1256, followed by a stack trace for a `NumberFormatException`
- F. The code does not compile.
- G. An uncaught exception is thrown.

**12.** What is printed by the following? (Choose all that apply)

```
1: public class Mouse {
2: public String name;
3: public void run() {
4: System.out.print("1");
5: try {
6: System.out.print("2");
7: name.toString();
8: System.out.print("3");
9: } catch (NullPointerException e) {
10: System.out.print("4");
11: throw e;
12: }
13: System.out.print("5");
14: }
15: public static void main(String[] args) {
16: Mouse jerry = new Mouse();
17: jerry.run();
18: System.out.print("6");
19: } }
```

- A. 1
  - B. 2
  - C. 3
  - D. 4
  - E. 5
  - F. 6
  - G. The stack trace for a `NullPointerException`
13. Which of the following statements are true? (Choose all that apply)
- A. You can declare a method with `Exception` as the return type.
  - B. You can declare any subclass of `Error` in the throws part of a method declaration.
  - C. You can declare any subclass of `Exception` in the throws part of a method declaration.
  - D. You can declare any subclass of `Object` in the throws part of a method declaration.
  - E. You can declare any subclass of `RuntimeException` in the throws part of a method declaration.
14. Which of the following can be inserted on line 8 to make this code compile? (Choose all that apply)
- ```
7: public void ohNo() throws IOException {  
8:    // INSERT CODE HERE  
9: }
```
- A. `System.out.println("it's ok");`
 - B. `throw new Exception();`
 - C. `throw new IllegalArgumentException();`
 - D. `throw new java.io.IOException();`
 - E. `throw new RuntimeException();`
15. Which of the following are unchecked exceptions? (Choose all that apply)
- A. `ArrayIndexOutOfBoundsException`
 - B. `IllegalArgumentException`
 - C. `IOException`
 - D. `NumberFormatException`
 - E. Any exception that extends `RuntimeException`
 - F. Any exception that extends `Exception`
16. Which scenario is the best use of an exception?
- A. An element is not found when searching a list.
 - B. An unexpected parameter is passed into a method.

- C. The computer caught fire.
- D. You want to loop through a list.
- E. You don't know how to code a method.

17. Which of the following can be inserted into `Lion` to make this code compile? (Choose all that apply)

```
class HasSoreThroatException extends Exception {}
class TiredException extends RuntimeException {}
interface Roar {
    void roar() throws HasSoreThroatException;
}
class Lion implements Roar { // INSERT CODE HERE
}
```

- A. `public void roar() {}`
- B. `public void roar() throws Exception {}`
- C. `public void roar() throws HasSoreThroatException {}`
- D. `public void roar() throws IllegalArgumentException {}`
- E. `public void roar() throws TiredException {}`

18. Which of the following are true? (Choose all that apply)

- A. Checked exceptions are allowed to be handled or declared.
- B. Checked exceptions are required to be handled or declared.
- C. Errors are allowed to be handled or declared.
- D. Errors are required to be handled or declared.
- E. Runtime exceptions are allowed to be handled or declared.
- F. Runtime exceptions are required to be handled or declared.

19. Which of the following can be inserted in the blank to make the code compile? (Choose all that apply)

```
public static void main(String[] args) {
    try {
        System.out.println("work real hard");
    } catch (_____ e) {
    } catch (RuntimeException e) {
    }
}
```

- A. `Exception`
- B. `IOException`
- C. `IllegalArgumentException`
- D. `RuntimeException`

- E. StackOverflowError
- F. None of the above.

20. What does the output of the following contain? (Choose all that apply)

```
12: public static void main(String[] args) {  
13:     System.out.print("a");  
14:     try {  
15:         System.out.print("b");  
16:         throw new IllegalArgumentException();  
17:     } catch (IllegalArgumentException e) {  
18:         System.out.print("c");  
19:         throw new RuntimeException("1");  
20:     } catch (RuntimeException e) {  
21:         System.out.print("d");  
22:         throw new RuntimeException("2");  
23:     } finally {  
24:         System.out.print("e");  
25:         throw new RuntimeException("3");  
26:     }  
27: }
```

- A. abce
- B. abde
- C. An exception with the message set to "1"
- D. An exception with the message set to "2"
- E. An exception with the message set to "3"
- F. Nothing; the code does not compile.

Note to the Reader: Throughout this index **boldfaced** page numbers indicate primary discussions of a topic. *Italicized* page numbers indicate illustrations.

A

- abstract classes
 - concrete classes from, 262–263
 - creating, 259
 - defining, 260–262
 - extending, 263–265
- abstract specifiers
 - interfaces, 267–269
 - methods, 168, 271–273
- access modifiers, 173
 - default, 175
 - description, 7
 - inheritance, 237
 - methods, 166–167
 - private, 173, 174
 - protected, 176, 176
 - public, 180–181
 - static, 181–188
- add() method, 130–131
- addition
 - dates, 142–143
 - precedence, 53–54
- ampersands (&) for logical operators, 64, 64
- ancestors in inheritance, 234
- “and” logical operator, 64, 64
- anonymous arrays, 120
- append() method, 115
- args parameters in main(), 7–8
- arithmetic operators, 53–55
- ArithmeticException class, 314
- ArrayIndexOutOfBoundsException class, 123, 314–315
- ArrayLists, 129
 - arrays from, 136–137
 - autoboxing, 136
 - creating, 129
 - methods, 130–134
 - sorting, 138
 - wrapper classes, 134–135
- arrays, 119
 - ArrayLists, 129
 - declaring, 121
 - indexes, 7, 120, 120

- multidimensional, 126–129, 127–128
- parameters, 7
- primitives, 119–121, 119–120
- reference variables, 121, 122
- searching, 125–126
- sorting, 124–125
- varargs, 126
- working with, 123–124

- arrow operator (->) for lambda expressions, 211–212, 212
- asList() method, 187–188
- assignment operators
 - compound, 62–63
 - overview, 60
 - precedence, 53
- assumed keywords in interfaces, 268
- asterisks (*)
 - comments, 4–5
 - packages, 10
- asymmetric arrays, 128, 128
- autoboxing
 - overloading methods, 193
 - type conversions, 136

B

- B prefix for binary, 22
- backed lists, 137
- base 10 numbering system, 22
- Beginning Java forum, 6
- binary number format, 22–23
- binary operators
 - arithmetic, 53–55
 - assignment, 60
 - casting, 60–61
 - compound assignment operators, 62–63
 - equality, 65–66
 - logical, 64–65, 64
 - numeric promotion, 55–57
 - relational, 63
- binary searches, 125–126
- bitwise operators, 64, 64

- blocks
 - description, 67
 - initialization order, 19–20
 - instance initializer, 18
 - scope in, 31–34
- bodies
 - lambda expressions, 212, 212
 - methods, 171–172
- boolean type
 - default initialization, 31
 - if statements, 70
 - logical operators, 57–58
 - size and range, 21
 - ternary operators, 71–72
 - while statements, 77
 - wrapper classes, 134–135
- braces ({})
 - blocks, 18–19, 32–33
 - if-then statements, 68
 - lambda expressions, 212, 212
 - methods, 171
 - try statements, 306–307
- brackets ([]) for arrays, 7, 119–122
- break statements
 - in loops, 88–90, 88
 - in switch, 74
- byte type
 - default initialization, 31
 - promotion rules, 56
 - size and range, 21
 - switch statements, 72–73
 - wrapper classes, 134–135
- bytecode, 6

C

- calling
 - constructors, 243–244
 - inherited class members, 244–245
 - static variables and methods, 182–183
- CamelCase names, 28–29
- capacity vs. size, 113–114, 114
- carets (^) for logical operators, 64, 64
- case sensitivity of names, 27
- case statements in switch, 73–76, 73
- casting
 - objects, 282–284
 - operators, 60–61
- catch blocks, 305–309, 305
- chaining
 - constructors, 201
 - methods, 110–111
 - StringBuilder, 112–113
- char type
 - default initialization, 31
 - promotion rules, 56
 - size and range, 21
 - switch statements, 72–73
 - wrapper classes, 134–135
- charAt() method, 106, 114, 135
- checked exceptions, 303, 305, 317
- child classes in inheritance, 234
- child packages, 10
- .class extension, 6
- class variables in default initialization, 30–31
- ClassCastException class, 315
- classes
 - abstract, 259–265
 - concrete, 262–263
 - description, 2
 - element ordering, 34–35
 - extending, 235–236, 235
 - fields and methods, 2–4
 - vs. files, 5
 - immutable, 207–208
 - inheritance. *See* inheritance
 - interfaces. *See* interfaces
 - packages, 9
 - paths, 15
 - subclasses, 319–321
 - wrapper, 134–136
- clear() method, 133
- closures. *See* lambda expressions
- code
 - blocks, 18–19
 - compiling, 6, 14–15
 - exam formatting, 16
- colons (:)
 - dates and times, 150
 - labels, 88
 - paths, 15
 - ternary operators, 71
- commas (,)
 - dates and times, 150
 - exception lists, 171
 - interface implementation, 267
 - parameter lists, 171
 - variable declarations, 26

- comments, 4
- comparators in arrays, 125
- compile-time constant values in switch statements, 73–76
- compiling code
 - extensions for, 6
 - and inheritance, 241–242
 - with packages, 14–15
- compound assignment operators, 62–63
- concatenating strings, 102–104
- concrete classes, 262–263
- conflicts in names, 12–13
- consistency of names, 28
- constant values in switch statements, 73–76
- constructors
 - calling, 243–244
 - chaining, 201
 - creating, 196
 - date and time, 141
 - default, 197–199
 - defining, 238
 - definition rules, 242–243
 - final fields, 202
 - initialization order, 202–204
 - objects, 17
 - overloading, 199–200
 - wrapper classes, 134–135
- contains() method
 - ArrayLists, 133
 - strings, 109–110
- continue statements, 90–91, 90
- control flow statements
 - break, 88–90, 88
 - continue, 90–91, 90
 - do-while, 78–80, 78
 - for, 80–83, 80, 85
 - for-each, 83–86, 83
 - if-then, 67–68, 67
 - if-then-else, 68–70, 69
 - labels, 87–89
 - nested loops, 87
 - switch, 72–76, 73
 - while, 76–77, 77, 79
- converting
 - ArrayLists to arrays, 136–137
 - number systems, 23
 - objects to wrapper classes, 135–136
- covariant return types, 248, 252
- curly braces ({})
 - blocks, 18–19, 32–33
 - if-then statements, 68

- lambda expressions, 212, 212
- methods, 171
- try statements, 306–307

D

- data types
 - in for statements, 82
 - promotion rules, 55–57
 - switch statements, 72–73
- Date class, 12–13
- dates
 - creating, 138–142
 - earlier versions, 141–142, 144, 150
 - formatting, 148–150
 - manipulating, 142–145
 - parsing, 151
 - periods, 145–147
- DateTimeFormatter class, 148–150
- dd format for dates and times, 150
- decimal numbering system, 22
- declare rules for exceptions, 303
- declaring
 - arrays, 121
 - methods, 166, 166
 - variables, 25
- decrement operators, 58
- default statements in switch, 72, 74
- defaults
 - access modifiers, 167, 175
 - constructors, 17, 197–199
 - interface methods, 274–278
 - packages, 13
 - variable initialization, 29
- defining
 - abstract classes, 260–262
 - interfaces, 267–269
- delete() method, 116
- deleteCharAt() method, 116
- descendants in inheritance, 234
- destroying objects
 - finalize(), 38–39
 - garbage collection, 36, 37–38
- diamond operator (<>), 130
- division
 - modulus operation, 54–55
 - precedence, 53–54
- do-while statements, 78–80, 78

- dollar signs (\$) for names, 27, 29
- double quotes (")
 - for spaces, 8
 - strings, 102–103
- double type
 - default initialization, 31
 - size and range, 21
 - wrapper classes, 134–135

E

- else operators, 68–70, 69
- encapsulation
 - description, 39
 - immutable classes, 207–208
 - overview, 205–206
- endsWith() method, 109
- enum type
 - dates, 140
 - support for, 72–73
 - switch statements, 72
- equal signs (=)
 - assignment operators, 60
 - compound assignment operators, 62–63
 - equality, 117–118
 - equality operators, 65–66
 - precedence, 53
 - relational operators, 63
 - unary operator, 57
 - variable declarations, 25
- equality
 - operators, 65–66
 - overview, 117–119
- equals() method
 - ArrayLists, 133–134
 - arrays, 121
 - strings, 109, 118–119
- equalsIgnoreCase() method, 109
- Error class, 302, 302, 305, 317–318
- exams, code formatting in, 16
- exception lists for methods, 171
- ExceptionInInitializerError class, 317
- exceptions
 - checked, 317
 - Error, 317–318
 - finally blocks, 307–309, 307
 - ignoring, 322
 - methods, 318–322
 - multiple, 311–313
 - printing, 321, 322
 - reasons, 300
 - vs. return codes, 301–302
 - roles, 300–301
 - runtime, 314–316
 - subclasses, 319–321
 - throwing, 304–305
 - try statements, 305–307, 305
 - types, 302–303, 302, 309
- exclamation points (!)
 - equality operators, 65–66
 - logical operator, 57–58
 - unary operator, 57
- “exclusive or” operator, 64, 64
- exit() method, 309
- extending
 - abstract classes, 263–265
 - classes, 235–236, 235
 - interfaces, 269

F

- false value in logical operators, 57
- fields
 - in classes, 35, 41
 - final, 202
 - initialization order, 19–20
 - overview, 2–4
 - reading and writing, 18
 - static, 181–182
- FileNotFoundException class, 317
- files vs. classes, 5
- final specifiers
 - abstract classes, 261
 - constants, 74
 - constructor fields, 202
 - interfaces, 267, 273–274
 - methods, 168, 256
- finalize() method, 38
- finally blocks, 307–309, 307
- float type
 - default initialization, 31
 - promotion rules, 55
 - size and range, 21
 - wrapper classes, 134–135
- for statements, 80–83, 80, 85
- for-each statements, 83–86, 83
- format() method, 148
- formatting dates and times, 148–150
- free store, 36

functional interfaces with lambda expressions, 214
 functional programming, 208–209
 functions. *See* methods

G

garbage collection, 36, 37–38
 gc() method, 36
 generics, 130
 getter methods in JavaBeans, 206
 getting variables, 18
 GMT (Greenwich Mean Time), 146
 greater than signs (>)
 diamond operator, 130
 lambda expressions, 211–212, 212
 relational operators, 63
 Greenwich Mean Time (GMT), 146

H

handle rules for exceptions, 303
 heap, 36
 hexadecimal number format, 22–23
 hh format for dates and times, 150
 hiding
 vs. overriding, 254–255
 static methods, 252–254
 variables, 257

I

identifiers, 27
 if-then statements, 67–68, 67
 if-then-else statements, 68–70, 69
 ignored returned values, 191
 IllegalArgumentException class, 315–316
 immutability of strings, 104–105
 immutable classes, creating, 207–208
 imports
 element ordering, 35
 packages, 9, 11–12
 static, 187–188
 “inclusive or” operator, 64, 64
 increment operators, 58
 indentation in if-then statements, 68

indexes
 arrays, 7, 120, 120
 strings, 105–107, 106
 indexOf() method, 106–107, 114
 infinite loops, 78
 infinite recursion, 317
 inheritance
 access modifiers, 237
 calling class members, 244–245
 compiler enhancements, 241–242
 constructors, 238, 242–244
 extending classes, 235–236, 235
 interfaces, 269–273, 277
 methods, 246
 object creation, 237–238, 238
 overview, 234, 235
 variables, 257
 initialization
 constructor order, 202–204
 default variables, 29
 object order, 19–20
 static, 186–187
 variables, 25–26
 initialization blocks in for statements, 80–82, 80
 initializer blocks, instances in, 18–19
 insert() method, 115
 instance members vs. static, 183–185
 instance variables
 default initialization, 30–31
 scope, 33
 instanceof operator, 63, 283
 instances
 creating, 17
 initialization order, 19
 initializer blocks, 18–19
 objects, 2
 instantiation process, 196
 int type
 default initialization, 31
 promotion rules, 55
 size and range, 21–22
 switch statements, 72–73
 wrapper classes, 134–135
 integrated development environment (IDE), 14
 interfaces
 abstract methods, 271–273
 with classes, 270
 default methods, 274–278
 defining, 267–269
 extending, 269
 implementing, 266, 266
 inheriting, 269–273

- lambda expressions, 214
- multiple inheritance, 277
- static methods, 278
- variables, 273–274
- intValue() method, 135
- IOException class, 317
- isEmpty() method, 132–133

J

- JAR files, 15
- Java Development Kit (JDK), 6
- .java extension, 6
- java.lang package, 11
- Java Runtime Environment (JRE), 6
- Java Virtual Machine (JVM), 6
- JavaBeans names, 205–206
- javac command, 15
- Javadoc comments
 - description, 4
 - key benefits, 39–40
- JDK (Java Development Kit), 6
- JRE (Java Runtime Environment), 6
- JVM (Java Virtual Machine), 6

K

- keywords, 3

L

- L suffix for long, 22
- labels, 87–89
- lambda expressions
 - example, 209–211
 - predicates, 214–215
 - syntax, 211–213, 212
 - variable access, 213
 - writing, 208–209
- length() method, 106, 114
- less than signs (<)
 - diamond operator, 130
 - relational operators, 63
- lists. *See* ArrayLists
- literal values
 - primitive types, 22–23
 - strings, 105, 117–118

- switch statements, 73–75
- local variables, 29
- LocalDate, 138–141, 146
- LocalDateTime, 139–141, 146
- LocalTime, 139–140
- logical operators, 64
 - complement, 57
 - overview, 64–65
 - precedence, 53
- long type
 - dates and times, 146
 - default initialization, 31
 - size and range, 21–22
 - wrapper classes, 134–135
- loops
 - arrays, 123
 - break statements, 88–90, 88
 - continue statements, 90–91, 90
 - do-while, 78–80, 78
 - for, 80, 85
 - for-each, 83–86, 83
 - infinite, 78
 - labels, 87–88
 - nested, 87
 - while, 76–77, 77, 79

M

- main() method, 6
- MAX_VALUE constant, 22
- MEDIUM format for dates and times, 149
- members in classes, 2
- memory
 - object destruction, 36, 37–38
 - reference types, 24, 24
- methods
 - abstract, 259, 271–273
 - access modifiers. *See* access modifiers
 - ArrayLists, 130–134
 - bodies, 171–172
 - chaining, 110–111
 - constructors. *See* constructors
 - dates and times, 144
 - declaring, 166, 166
 - defining, 3
 - designing, 166, 166
 - exception lists, 171
 - exceptions thrown by, 318–322
 - final, 256
 - hiding, 252–254
 - interfaces, 271–279

- main(), 6
- names, 170
- optional specifiers, 168–169
- overloading, 191
- overriding, 246, 319–321
- overriding vs. hiding, 254–255
- overview, 2–4
- parameter lists, 171
- passing data, 188–191, 190
- polymorphism, 287–288
- redeclaring, 251–252
- return types, 169–170
- signatures, 3, 7, 166, 166
- static, 181–182
- StringBuilder class, 114–117
- strings, 105–109
- varargs, 172–173
- virtual, 284–285
- minus signs (-)
 - compound assignment operators, 62
 - increment operators, 58–59
 - lambda expressions, 211–212, 212
 - negation operator, 57–58
 - unary operator, 57
- mm format for dates and times, 150
- MMMM format for dates and times, 150
- modulus operator (%)
 - overview, 54–55
 - precedence, 53–54
- multidimensional arrays, 126–129, 127–128
- multiple exceptions, 311–313
- multiple inheritance
 - description, 234–235, 235
 - interfaces, 271–273, 277
- multiple-line comments, 4–5
- multiple variables, declaring, 26
- multiplication, 53–54
- mutability of StringBuilder, 112–113

N

- names
 - conflicts, 12–13
 - constructors, 17
 - identifiers, 27
 - JavaBeans, 205–206
 - methods, 170
 - packages, 10, 12–13
- native specifiers, 168
- negation operator, 57
- nested loops, 87

- new keyword for constructors, 17
- no-argument constructors, 198
- NoClassDefFoundError class, 318
- not equals operators, 65–66
- now() method, 139
- null values
 - autoboxing, 136
 - reference types, 24
- NullPointerException class, 303, 316
- NumberFormatException class, 316
- numbering systems, 22–23
- numeric promotion, 55–57

O

- object-oriented languages, 39
- objects
 - casting, 282–284
 - comparing, 66
 - constructors, 17
 - creating, 16, 237–238, 238
 - description, 2
 - destroying, 36–39, 37–38
 - initialization order, 19–20
 - instance initializer blocks, 18–19
 - polymorphism, 281–282, 282
 - primitive types, 20–23
 - reading and writing fields, 18
 - vs. references, 36, 37
- octal number format, 22
- ofLocalized methods, 149
- operators
 - arithmetic, 53–55
 - assignment, 60, 62
 - casting, 60–61
 - equality, 65–66
 - logical, 64–65, 64
 - numeric promotion, 55–57
 - order, 52
 - overview, 52
 - relational, 63
 - ternary, 71
 - unary, 57–59
- optional specifiers, 168–169
- “or” operator, 64, 64
- order
 - class elements, 34–35
 - constructor initialization, 202–204
 - field initialization, 19–20
 - operators, 52

- overflow, 61
- overloading methods, 191
 - autoboxing, 193
 - constructors, 199–200
 - vs. overriding, 248
 - overview, 194–196
 - primitive types, 194
 - reference types, 193–194
 - varargs, 192–193
- overriding
 - vs. hiding, 254–255
 - methods, 246, 319–321
 - vs. overloading, 248
 - polymorphism, 287–288

P

- package private access modifiers, 167, 175
- packages
 - compiling code with, 14–15
 - creating, 13–14
 - element ordering, 35
 - names, 10, 12–13
 - overview, 9
 - redundant imports, 11–12
 - wildcards, 10–11
- parameters
 - main(), 7
 - methods, 3, 171
 - overloading methods, 192–193, 199
 - polymorphic, 285–286
- parent classes in inheritance, 234
- parentheses ()
 - lambda expressions, 212–213, 212
 - methods, 167
 - precedence, 54
- parse() method, 151
- parseInt() method, 135
- parsing dates and times, 151
- pass-by-reference languages, 190–191
- pass-by-value languages, 188–190, 190
- passing data in methods, 188–191, 190
- paths for classes, 15
- percent signs (%) for modulus, 54
- Period class, 146–147
- periods in dates and times, 145–147
- platform independence, 39–40

- plus methods for dates, 144
- plus signs (+)
 - compound assignment operators, 62
 - increment operators, 58–59
 - string concatenation, 103
 - unary operator, 57
- pointers, 24
- polymorphism
 - casting objects, 282–284
 - method overriding, 287–288
 - objects vs. references, 281–282, 282
 - overview, 279
 - parameters, 285–286
 - virtual methods, 284–285
- pools, strings, 105
- post-decrement operators, 58–59
- post-increment operators, 58–59
- pre-decrement operators, 58–59
- pre-increment operators, 58–59
- precedence of operators, 52
- predicates in lambda expressions, 214–215
- primitive types
 - arrays, 119–121, 119–120
 - casting, 60–61
 - overloading methods, 194
 - overview, 20–23
 - vs. reference, 25
- printing exceptions, 321, 322
- private access modifiers
 - abstract classes, 261
 - inheritance, 237
 - interfaces, 267–269
 - methods, 167
 - overview, 173, 174
- private methods, redeclaring, 251–252
- procedural languages, 39
- procedures. *See* methods
- promotion, numeric, 55–57
- properties in JavaBeans, 205
- protected access modifiers
 - interfaces, 267–269
 - methods, 167
 - overview, 176, 176
- public access modifiers
 - inheritance, 237
 - interfaces, 267–268, 273–274
 - methods, 167
 - overview, 7, 180–181

Q

question marks (?) for ternary operators, 71
 quotes (")
 for spaces, 8
 strings, 102–103

R

ranges of primitive types, 21
 readability
 if-then statements, 68
 literal values, 23
 names, 28
 polymorphic parameters, 286
 reading fields, 18
 recursion, infinite, 317
 recursive functions, 247
 redeclaring private methods, 251–252
 redundant package imports, 11–12
 references
 in arrays, 121, 122
 comparing, 66
 methods, 189–191, 190
 vs. objects, 36, 37
 overloading methods, 194
 overview, 24, 24
 polymorphism, 281–282, 282
 vs. primitive, 25
 relational operators
 overview, 63
 precedence, 53
 remainder operator (%)
 overview, 54–55
 precedence, 53–54
 remove() method, 131–132
 replace() method, 110
 reserved words, 27
 return codes vs. exceptions, 301–302
 return statement in methods, 169
 return types and values
 constructors, 17
 covariant, 248, 252
 description, 7
 ignored, 191
 immutable classes, 207–208

 methods, 169–170
 reverse() method, 116
 robustness, 40
 runtime exceptions, 314–316
 RuntimeException class, 302–303, 302, 305

S

scope
 in for statements, 81
 variables, 31–34
 searching arrays, 125–126
 security, 40
 semicolons (;)
 class paths, 15
 in for statements, 81
 lambda expressions, 212–213, 212
 variable declarations, 27
 set() method, 132
 setter methods in JavaBeans, 206
 setting variables, 18
 shift operators, 53
 short-circuit logical operators
 description, 64–65
 precedence, 53
 SHORT format for dates and times, 149
 short type
 default initialization, 31
 promotion rules, 56
 size and range, 21
 switch statements, 72–73
 wrapper classes, 134–135
 signatures in methods, 3, 7, 166, 166
 simplicity, 40
 single inheritance, 234, 235
 single-line comments, 4
 size
 vs. capacity, 113–114, 114
 primitive types, 21
 size() method, 132–133
 slashes (/)
 comments, 4–5
 division, 53
 precedence, 54
 sorting
 ArrayLists, 138

- arrays, 124–125
- spaces in arguments, 8
- square brackets ([]) for arrays, 7, 119–122
- StackOverflowError class, 318
- startsWith() method, 109
- statements
 - do-while, 78–80, 78
 - for, 80–83, 80, 85
 - for-each, 83–86, 83
 - if-then, 67–68, 67
 - if-then-else, 68–70, 69
 - overview, 66
 - switch, 72–76, 73
 - while, 76–77, 77, 79
- static keyword, 7
- static methods, hiding, 252–254
- static specifiers
 - calling variables and methods, 182–183
 - designing, 181–182
 - imports, 187–188
 - initialization, 186–187
 - vs. instance, 183–185
 - interfaces, 273–274, 278
 - methods, 168
 - variables, 185
- strictfp specifiers, 168
- String class, 102
- StringBuffer class vs, 117
- StringBuilder class, 111–112
 - creating, 113–114
 - methods, 114–117
 - mutability and chaining, 112–113
 - vs. StringBuffer class, 117
- strings
 - as arrays, 119
 - concatenating, 102–104
 - description, 3, 102
 - immutability, 104–105
 - methods, 105–109
 - pools, 105
 - StringBuilder class, 111–117
 - switch statements, 73
- subclasses with exceptions, 319–321
- substring() method, 107–108, 114
- subtraction
 - days, 145
 - precedence, 53–54
- super() keyword, 239–243, 246
- super keyword, 246–247

- switch statements, 72
 - compile-time constant values, 73–76
 - data types, 72–73, 73
- synchronized specifiers, 168
- System.gc() method, 36

T

- ternary operators
 - overview, 71
 - precedence, 53
- this() keyword, 238
- Throwable class, 302, 302
- throwing exceptions, 304–305
- times
 - creating, 138–142
 - formatting, 148–150
 - manipulating, 142–145
 - parsing, 151
 - periods, 145–147
- toEpochDay() method, 146
- toEpochTime() method, 146
- toLowerCase() method, 108
- toString() method, 105, 116–117
- toUpperCase() method, 108
- trim() method, 110
- true value in logical operators, 57
- truth tables, 64, 64
- try statements, 305–307, 305
- two-multidimensional arrays, 127

U

- unary operators
 - increment and decrement, 58–59
 - logical complement and negation, 57–58
 - precedence, 53
 - promotion rules, 56
 - working with, 57
- unchecked exceptions, 302
- underflow, 61
- underscores (_)
 - in literal values, 23
 - names, 27
- Unicode character set, 29
- update statements in for statements, 80–82, 80

V

- valueOf() method, 135
- variable argument (varargs) lists, 7–8
 - ArrayLists, 137
 - arrays, 126
 - methods, 172–173
 - overloading methods, 192–193
- variables
 - assignment operators, 60, 62–63
 - declaring, 25
 - default initialization, 29
 - defining, 3
 - description, 2
 - in for statements, 81–83
 - for-each statements, 83
 - hiding, 257
 - inheriting, 257
 - interfaces, 273–274
 - lambda expression access, 213
 - reading and writing, 18
 - scope, 31–34
 - static, 185
- vertical bars (|) for logical operators, 64, 64
- virtual methods, 284–285
- void type, 7
 - description, 3
 - methods, 169

W

- while statements, 76–77, 77, 79
- wildcards
 - JAR files, 15
 - packages, 10–11
- wrapper classes
 - ArrayLists, 134–135
 - converting to, 135–136
 - switch statements, 73
- “write once, run everywhere”, 40
- writing fields, 18

X

- X for hexadecimal, 22

Z

- ZonedDateTime, 139

Free Online Learning Environment

Register on Sybex.com to gain access to the free online interactive learning environment and test bank to help you study for your OCA Java SE 8 Programmer I certification.

The online test bank includes:

- Assessment Test to help you focus your study to specific objectives
- Chapter Tests to reinforce what you learned
- Practice Exams to test your knowledge of the material
- Electronic Flashcards to reinforce your learning and provide last-minute test prep before the exam
- Searchable Glossary gives you instant access to the key terms you'll need to know for the exam

Go to www.sybex.com/go/ocejavase8 to register and gain access to this comprehensive study tool package.



 **SYBEX**
An Imprint of  **WILEY**

WILEY END USER LICENSE AGREEMENT

Go to www.wiley.com/go/eula to access Wiley's ebook EULA.